

Universidad Mariano Gálvez
Ingeniería en sistemas
Sede boca del monte
Séptimo semestre
Sección "A"
Análisis de sistemas 1
Ing. Mélvín Cali



Fase #3

Elmer Yovani Rodríguez de Paz 7690-22-11804
Fredy Roberto Orozco Alvarado 7690-22-16980

Guatemala 21/05/2026

SISTEMA LA ESPERANZA

Plataforma de Gestión Agrícola

Fase#3

Sistema de Gestión y Comercialización para una
Comunidad Agrícola Rural — "La Esperanza"

Arquitectura de componentes y patrones

1. Visión General de la Arquitectura

El Sistema La Esperanza implementa una arquitectura de 5 capas que separa claramente las responsabilidades entre la interfaz de usuario, el enrutamiento, la lógica de negocio, el acceso a datos y la infraestructura. Esta separación facilita el mantenimiento, las pruebas independientes y la escalabilidad horizontal.

#	Capa	Descripción y tecnología principal
1	Presentación	React 18 + Vite. SPA con componentes funcionales. Zustand para estado global. Axios para HTTP.
2	Gateway / Proxy	Nginx. Sirve el build estático del frontend y redirige /api/* al backend Node.js.
3	Lógica de Negocio	Node.js 20 + Express. Arquitectura interna en 3 sub-capas: Routes → Controllers → Services. Los controladores acceden a Prisma directamente, excepto auth que usa una capa de servicio.
4	Datos	Prisma ORM + PostgreSQL 16. 12 tablas, 5 enums, triggers automáticos y 2 vistas.
5	Infraestructura	Docker Compose orquesta los 3 contenedores. Variables de entorno centralizadas en .env.

2. Descripción de Componentes

Frontend

Componente	Tecnología	Responsabilidad
React SPA	React 18	Renderiza la interfaz de usuario. Gestiona el árbol de componentes y el ciclo de vida. 10 páginas + componentes compartidos.
Enrutador	React Router v6	Define rutas públicas y privadas. Implementa guardias de autenticación y restricción por rol. Maneja navegación sin recarga de página.
Estado Global	Zustand	Almacena el estado de autenticación (usuario, token) con persistencia en localStorage. Reemplaza prop drilling entre componentes.
Cliente HTTP	Axios	Instancia centralizada con baseURL configurable. Interceptores que inyectan JWT en cabeceras y manejan errores 401 globalmente.
Build Tool	Vite 5	Servidor de desarrollo con HMR. Produce el build optimizado de producción con code splitting y tree shaking.
Estilos	CSS Puro	Sistema de diseño con variables CSS. Paleta verde agrícola + dorado. Diseño responsivo sin dependencias externas de CSS.

Gateway

Componente	Tecnología	Responsabilidad
Proxy inverso	Nginx Alpine	Sirve los archivos estáticos del build de React. Redirige todas las peticiones a /api/* hacia el contenedor backend en el puerto 3000.
SPA Fallback	Nginx try_files	Redirige todas las rutas desconocidas a index.html para que React Router maneje la navegación en el lado del cliente.
Cache estático	Nginx Cache	Headers Cache-Control con expiración de 1 año para assets estáticos (JS, CSS, imágenes) identificados por hash de contenido.

Backend — API REST

Componente	Tecnología	Responsabilidad
Servidor HTTP	Express.js	Framework minimalista para Node.js. Define las rutas, aplica middlewares globales (CORS, JSON parser, Morgan) y maneja errores.
Autenticación	JWT + bcryptjs	Genera y verifica tokens JWT con clave secreta configurable. Hashea contraseñas con bcrypt (factor de coste 10). Middleware verificarToken().
Control de Roles	RBAC Middleware	Función soloRoles(...roles) que verifica el rol del usuario autenticado. Retorna 403 si el rol no está autorizado para el endpoint.
ORM	Prisma Client	Generado desde schema.prisma. Provee una API tipo-segura para consultas, mutaciones, transacciones y relaciones entre modelos.
Logging	Morgan	Registra cada petición HTTP: método, ruta, código de respuesta y tiempo. Útil para depuración en desarrollo.

Módulos de la API

Componente	Tecnología	Responsabilidad
auth	/api/v1/auth	Registro, login y obtención del perfil autenticado. Genera tokens JWT y crea el perfil de rol correspondiente.
publicaciones	/api/v1/publicaciones	CRUD completo de ofertas de venta. Filtros por departamento, precio, producto. Control de estado y propiedad.
negociaciones	/api/v1/negociaciones	Gestión del flujo de negociación. Validación de transiciones de estado. Privacidad entre participantes.
mensajes	(en negociaciones)	Mensajería interna por negociación. Solo accesible por los dos participantes. Marcado de mensajes leídos.
entregas	/api/v1/entregas	Registro y seguimiento de entregas. Implementa la lógica de confirmación doble con trigger automático.
pagos	/api/v1/pagos	Registro de transacciones económicas. Soporte para múltiples métodos de pago y estados del ciclo de vida.
notificaciones	/api/v1/notificaciones	Alertas generadas automáticamente por eventos del sistema.

Componente	Tecnología	Responsabilidad
		Marcado individual y masivo.
productores	/api/v1/productores	Perfiles de productores. Listado con filtros. Actualización de datos agrícolas.
compradores	/api/v1/compradores	Perfiles de compradores. Datos comerciales (razón social, NIT, tipo).
asociaciones	/api/v1/asociaciones	Gestión de asociaciones de agricultores y sus miembros productores.
productos	/api/v1/productos	Catálogo de productos agrícolas. Lectura pública. Creación restringida a rol asociación.

Base de Datos

Componente	Tecnología	Responsabilidad
PostgreSQL 16	postgres:16-alpine	Motor relacional principal. 12 tablas, 5 tipos ENUM, índices en campos de filtrado frecuente.
Triggers	PL/pgSQL	trg_update_timestamp: actualiza updated_at automáticamente. trg_doble_confirmacion: cierra negociación al confirmar ambas partes.
Vistas	SQL Views	v_publicaciones_activas: join optimizado de publicaciones con productor y producto. v_negociaciones_resumen: participantes de cada negociación.
Migraciones	Prisma Migrate	Historial versionado de cambios al esquema. Reproducible en cualquier entorno con 'prisma migrate deploy'.
Seed	prisma/seed.js	Carga el catálogo inicial de 12 productos y crea 3 cuentas de prueba (productor, comprador, asociación).

3. Patrones de Diseño Aplicados

A continuación se documentan los patrones de diseño implementados en el sistema, con la justificación técnica de su adopción.

Repository Pattern (Capa de Datos)

Descripción:

Abstrae el acceso a la base de datos detrás de una interfaz. Los servicios nunca llaman a Prisma directamente; siempre lo hacen a través del repositorio correspondiente.

Justificación:

- Desacopla la lógica de negocio del motor de base de datos. Cambiar de PostgreSQL a otro motor no afecta a los servicios.

- Facilita el testing: el repositorio puede ser mockeado en pruebas unitarias.
- Centraliza las consultas repetitivas en un solo lugar.

Implementación en el proyecto:

```
// backend/src/repositories/publicaciones.repository.js
const prisma = new PrismaClient();

async function findAll(where, pagination) {
  return prisma.publicacion.findMany({ where, ...pagination });
}

async function findById(id) {
  return prisma.publicacion.findUnique({ where: { id } });
}

module.exports = { findAll, findById };
```

CONSIDERACIÓN: En la implementación actual los repositorios están integrados en los controladores para simplificar la estructura. La separación completa se aplica en proyectos de mayor escala.

Middleware Pattern (Cadena de Responsabilidad)

Descripción:

Las solicitudes HTTP pasan por una cadena de middlewares antes de llegar al controlador final. Cada middleware tiene una responsabilidad específica y puede detener o continuar la cadena.

Justificación:

- Separación de preocupaciones: autenticación, logging, parsing y control de roles son independientes.
- Reutilización: los middlewares `verificarToken()` y `soloRoles()` se aplican a cualquier ruta sin duplicar código.
- Composabilidad: se pueden combinar múltiples middlewares en cualquier orden.

Implementación en el proyecto:

```
// Cadena de middlewares en una ruta protegida
router.post('/',
  verificarToken,           // 1. ¿Tiene token válido?
  soloRoles('productor'),   // 2. ¿Es productor?
  publicacionesController.crear // 3. Ejecutar lógica
```

```
);
```

Singleton Pattern (Instancia de Prisma)

Descripción:

Se crea una única instancia del PrismaClient compartida entre todos los módulos del backend, evitando múltiples conexiones innecesarias al pool de la base de datos.

Justificación:

- PostgreSQL tiene un límite de conexiones simultáneas. Una instancia por módulo agota el pool rápidamente.
- Reduce la latencia de conexión: la misma instancia reutiliza las conexiones del pool.
- Facilita la gestión del ciclo de vida: shutdown limpio con `$disconnect()`.

Implementación en el proyecto:

```
// En el prototipo actual cada módulo crea su instancia directamente (patrón simplificado)
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
module.exports = prisma; // Misma instancia en toda la app
```

Observer Pattern (Notificaciones automáticas)

Descripción:

Cuando ocurre un evento relevante en el sistema (nueva negociación, mensaje recibido, confirmación de entrega), el backend genera automáticamente una notificación para el usuario afectado sin que el emisor conozca a los suscriptores.

Justificación:

- Desacopla el productor del evento (quien crea la negociación) del consumidor (quien recibe la notificación).
- Extensible: agregar nuevos tipos de notificación no requiere modificar el código del evento original.
- Mejora la experiencia de usuario con alertas en tiempo real sin necesidad de polling.

Implementación en el proyecto:

```
// Al crear una negociación, se notifica automáticamente al productor
```

```
const negociacion = await prisma.negociacion.create({ data });

// Observer: genera notificación sin que el comprador lo sepa
await prisma.notificacion.create({
  data: {
    usuario_id: productor.usuario_id,
    tipo: 'nueva_negociacion',
    titulo: 'Nueva solicitud de negociación',
    mensaje: `Interesado en: ${publicacion.titulo}`,
    referencia_id: negociacion.id
  }
});
```

State Pattern (Máquina de Estados de Negociación)

Descripción:

El estado de una negociación sigue un flujo definido con transiciones válidas. El sistema valida que cada cambio de estado sea una transición permitida desde el estado actual.

Justificación:

- Previene estados incoherentes: no se puede pasar de 'completada' a 'pendiente'.
- La lógica de transición está centralizada en un único mapa, fácil de auditar y extender.
- Cada estado puede tener comportamiento diferente (precio obligatorio al aceptar, trigger al completar, etc.).

Implementación en el proyecto:

```
// Mapa de transiciones válidas
const TRANSICIONES = {
  pendiente: ['en_proceso', 'aceptada', 'rechazada', 'cancelada'],
```

```
en_proceso: ['aceptada', 'rechazada', 'cancelada'],
aceptada: ['completada', 'cancelada'],
rechazada: [], // Estado terminal
completada: [], // Estado terminal
cancelada: [], // Estado terminal
};

// Validación antes de cambiar estado
if (!TRANSICIONES[neg.estado].includes(nuevoEstado))
  throw new Error('Transición de estado inválida');
```

Facade Pattern (Controladores como Fachada)

Descripción:

Los controladores actúan como fachada entre las rutas HTTP y la lógica interna. El cliente HTTP solo conoce los endpoints; no sabe nada de Prisma, validaciones de negocio ni estructuras internas.

Justificación:

- Simplifica la interfaz externa: el frontend solo hace peticiones HTTP estándar.
- Oculta la complejidad interna: lógica de roles, triggers, notificaciones son invisibles al cliente.
- Facilita cambios internos sin afectar la interfaz pública de la API.

Template Method en Triggers (Base de Datos)

Descripción:

Los triggers de PostgreSQL siguen una plantilla común: verificar condición → ejecutar lógica → actualizar estado. Este patrón se repite en el trigger de updated_at y en el de confirmación doble.

Implementación en el proyecto:

```
-- Template: verificar → actualizar
CREATE OR REPLACE FUNCTION fn_verificar_doble_confirmacion()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.confirmado = TRUE THEN      -- 1. Verificar condición
        -- Contar confirmaciones actuales
        IF total_confirmaciones >= 2 THEN -- 2. Condición cumplida
            UPDATE entregas SET estado = 'entregado'; -- 3. Actualizar
            UPDATE negociaciones SET estado = 'completada';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

4. Decisiones Arquitectónicas Clave

Decisión	Alternativa considerada	Justificación de elección
Zustand vs Redux	Redux Toolkit	Zustand es 10x más liviano y no requiere boilerplate. Suficiente para el estado de autenticación del sistema.
Prisma vs Sequelize	Sequelize ORM	Prisma genera tipos TypeScript, tiene migraciones robustas y una DX superior. Integra mejor con PostgreSQL.
JWT stateless vs Sesiones	Express-session	JWT es stateless: no requiere almacenamiento en servidor, escala horizontalmente sin sesiones compartidas.
Docker Compose vs k8s	Kubernetes	Compose es suficiente para la escala del sistema. K8s agrega complejidad innecesaria para un despliegue simple.
PostgreSQL vs MongoDB	MongoDB	El modelo de datos es relacional con integridad referencial fuerte. SQL es la elección natural.
Vite vs Create React App	Create React App	Vite es 10-100x más rápido en desarrollo. CRA está deprecado. Vite es el estándar actual.

Casos de prueba

1. Resumen de Resultados

Total de casos ejecutados: 37 | Aprobados: 37 | Tasa de éxito: 100%

Módulo	Total	Aprob.	Fallidos	Éxito
Autenticación	6	6	0	100%
Publicaciones	7	7	0	100%
Negociaciones	7	7	0	100%
Mensajería	4	4	0	100%
Entregas y Confirmación Doble	6	6	0	100%
Pagos	4	4	0	100%
Notificaciones	3	3	0	100%
TOTAL	37	37	0	100%

2. Casos de Prueba Detallados

Modulo 1: Autenticación

CP-01 Registro exitoso de productor		
Módulo	Autenticación	
Precondición	El sistema está operativo. El teléfono '4256-0001' no está registrado.	
Pasos	1. Navegar a la pantalla de registro. 2. Ingresar nombre: 'Juan Prueba', teléfono: '4256-0001', contraseña: 'Test1234!', rol: 'productor'. 3. Hacer clic en 'Registrarse'.	
Resultado esperado	El sistema crea el usuario, retorna código 201 y redirige al dashboard del productor.	
Resultado real	El sistema registra al usuario y redirige correctamente al dashboard.	APROBADO

CP-02 Registro con teléfono duplicado		
Módulo	Autenticación	
Precondición	Existe un usuario registrado con el teléfono '4256-0001'.	
Pasos	1. Navegar a la pantalla de registro. 2. Ingresar teléfono: '4256-0001' con datos válidos. 3. Hacer clic en 'Registrarse'.	
Resultado esperado	El sistema retorna código 409 y muestra: 'El número de teléfono ya está registrado'.	
Resultado real	El sistema muestra el mensaje de error. No crea duplicado.	APROBADO

CP-03 Login con credenciales válidas		
Módulo	Autenticación	
Precondición	Existe el usuario con teléfono '4256-0001' y contraseña 'Test1234!'.	
Pasos	1. Navegar a la pantalla de login. 2. Ingresar teléfono: '4256-0001' y contraseña: 'Test1234!'. 3. Hacer clic en 'Iniciar sesión'.	
Resultado esperado	El sistema autentica al usuario, retorna token JWT y redirige al dashboard.	
Resultado real	Login exitoso. Token generado y almacenado. Redirección correcta.	APROBADO

CP-04 Login con contraseña incorrecta		
Módulo	Autenticación	
Precondición	Existe el usuario con teléfono '4256-0001'.	
Pasos	1. Navegar a la pantalla de login. 2. Ingresar teléfono: '4256-0001' y contraseña: 'Claveincorrecta'. 3. Hacer clic en 'Iniciar sesión'.	
Resultado esperado	El sistema retorna código 401 y muestra: 'Credenciales incorrectas'.	
Resultado real	Se muestra el mensaje de error. No se genera token.	APROBADO

CP-05 Acceso a endpoint protegido sin token		
Módulo	Autenticación	
Precondición	Ninguna sesión activa.	
Pasos	1. Hacer petición GET a /api/v1/negociaciones sin header Authorization.	
Resultado esperado	El sistema retorna código 401 con mensaje 'Token requerido'.	
Resultado real	API retorna 401. Acceso denegado correctamente.	APROBADO

CP-06 Acceso con token expirado		
Módulo	Autenticación	
Precondición	Token JWT generado hace más de 7 días (expirado).	
Pasos	1. Hacer petición GET a /api/v1/auth/me con el token expirado en el header Authorization.	
Resultado esperado	El sistema retorna código 401 con mensaje 'Token expirado'.	
Resultado real	API retorna 401. Token expirado detectado correctamente.	APROBADO

Modulo 2: Publicaciones

CP-07 Crear publicación con datos válidos		
Módulo	Publicaciones	
Precondición	Usuario autenticado con rol 'productor'. Existe el producto ID=1 en el catálogo.	
Pasos	1. Autenticarse como productor. 2. Hacer POST a /api/v1/publicaciones con: producto_id=1, titulo='Maíz blanco', cantidad_disponible=50, precio_unitario=120.00, unidad_medida='quintal'.	
Resultado esperado	Publicación creada con estado 'activa' y código 201.	
Resultado real	Publicación creada exitosamente. Estado 'activa' por defecto.	APROBADO

CP-08 Crear publicación sin campos requeridos		
Módulo	Publicaciones	
Precondición	Usuario autenticado con rol 'productor'.	
Pasos	1. Hacer POST a /api/v1/publicaciones omitiendo el campo 'titulo'.	
Resultado esperado	El sistema retorna código 400 indicando los campos faltantes.	
Resultado real	API retorna 400 con mensaje de validación.	APROBADO

CP-09 Comprador no puede crear publicación		
Módulo	Publicaciones	
Precondición	Usuario autenticado con rol 'comprador'.	
Pasos	1. Hacer POST a /api/v1/publicaciones con datos válidos usando token de comprador.	

Resultado esperado	El sistema retorna código 403: 'Solo productores pueden publicar'.	
Resultado real	API retorna 403. Control de roles funciona correctamente.	APROBADO

CP-10 Listar publicaciones con filtro por departamento		
Módulo	Publicaciones	
Precondición	Existen publicaciones con departamento='Chiquimula' y otras con departamento='Guatemala'.	
Pasos	1. Hacer GET a /api/v1/publicaciones?departamento=Chiquimula.	
Resultado esperado	Solo se retornan publicaciones del departamento 'Chiquimula'.	
Resultado real	Filtro aplicado correctamente.	APROBADO

CP-11 Actualizar estado de publicación a 'pausada'		
Módulo	Publicaciones	
Precondición	Existe publicación ID=3 del productor autenticado con estado 'activa'.	
Pasos	1. Autenticarse como el productor propietario. 2. Hacer PATCH a /api/v1/publicaciones/3/estado con body: { estado: 'pausada' }.	
Resultado esperado	Estado actualizado a 'pausada' y código 200.	
Resultado real	Estado actualizado correctamente.	APROBADO

CP-12 Productor no puede editar publicación ajena		
Módulo	Publicaciones	
Precondición	Publicación ID=5 pertenece al productor B. Se autentica el productor A.	
Pasos	1. Autenticarse como productor A. 2. Hacer PUT a /api/v1/publicaciones/5 con datos de actualización.	
Resultado esperado	El sistema retorna código 403: 'Sin permiso para editar esta publicación'.	
Resultado real	API retorna 403. Control de propiedad funciona.	APROBADO

CP-13 Búsqueda por rango de precio		
Módulo	Publicaciones	
Precondición	Existen publicaciones con precios entre Q50 y Q300.	
Pasos	1. Hacer GET a /api/v1/publicaciones?precio_min=100&precio_max=200.	
Resultado esperado	Solo publicaciones con precio entre 100 y 200.	
Resultado real	Filtro por rango aplicado correctamente.	APROBADO

Medulo 3: Negociaciones

CP-14 Comprador inicia negociación		
Módulo	Negociaciones	
Precondición	Existe publicación ID=1 activa. Usuario autenticado como comprador.	
Pasos	1. Autenticarse como comprador. 2. Hacer POST a /api/v1/negociaciones con publicacion_id=1, cantidad_solicitada=10.	
Resultado esperado	Negociación creada con estado 'pendiente' y código 201.	
Resultado real	Negociación creada. Notificación generada al productor.	APROBADO

CP-15 Productor no puede iniciar negociación		
Módulo	Negociaciones	
Precondición	Usuario autenticado con rol 'productor'.	
Pasos	1. Hacer POST a /api/v1/negociaciones con token de productor.	

Resultado esperado	Código 403: 'Solo compradores pueden iniciar negociaciones'.	
Resultado real	API retorna 403 correctamente.	APROBADO

CP-16 Productor acepta negociación con precio		
Módulo	Negociaciones	
Precondición	Negociación ID=2 en estado 'pendiente'. Productor es participante.	
Pasos	1. Autenticarse como productor. 2. Hacer PATCH a /api/v1/negociaciones/2/estado con estado='aceptada', precio_acordado=115.00.	
Resultado esperado	Estado cambia a 'aceptada', precio guardado, código 200.	
Resultado real	Negociación aceptada. Precio guardado. Notificación enviada.	APROBADO

CP-17 Rechazar negociación		
Módulo	Negociaciones	
Precondición	Negociación ID=3 en estado 'pendiente'.	
Pasos	1. Autenticarse como productor. 2. PATCH /api/v1/negociaciones/3/estado con estado='rechazada'.	
Resultado esperado	Estado cambia a 'rechazada', código 200.	
Resultado real	Negociación rechazada correctamente.	APROBADO

CP-18 Transición de estado inválida		
Módulo	Negociaciones	
Precondición	Negociación ID=4 en estado 'completada'.	
Pasos	1. PATCH /api/v1/negociaciones/4/estado con estado='pendiente'.	
Resultado esperado	Código 400: 'Transición de estado inválida'.	
Resultado real	API retorna 400. No permite retroceder estado.	APROBADO

CP-19 No-participante no puede ver negociación		
Módulo	Negociaciones	
Precondición	Negociación ID=5 pertenece a productor A y comprador B. Se autentica usuario C.	
Pasos	1. Autenticarse como usuario C. 2. Hacer GET a /api/v1/negociaciones/5.	
Resultado esperado	Código 403: 'No es participante de esta negociación'.	
Resultado real	API retorna 403. Privacidad garantizada.	APROBADO

CP-20 Negociación duplicada		
Módulo	Negociaciones	
Precondición	Ya existe negociación activa del comprador A sobre publicación ID=1.	
Pasos	1. Autenticarse como comprador A. 2. POST /api/v1/negociaciones con publicacion_id=1.	
Resultado esperado	Código 409: 'Ya existe una negociación activa para esta publicación'.	
Resultado real	API retorna 409. No se crea duplicado.	APROBADO

Modulo 4: Mensajería

CP-21 Enviar mensaje en negociación		
Módulo	Mensajería	
Precondición	Negociación ID=2 activa. Usuario es participante.	
Pasos	1. Autenticarse como comprador participante. 2. POST /api/v1/negociaciones/2/mensajes con contenido: '¿Podría entregar el viernes?'.	
Resultado	Mensaje creado con código 201. Productor puede verlo.	

esperado		
Resultado real	Mensaje guardado. Notificación generada para el productor.	APROBADO

CP-22 No-participante no puede enviar mensajes		
Módulo	Mensajería	
Precondición	Negociación ID=2. Usuario C no es participante.	
Pasos	1. Autenticarse como usuario C. 2. POST /api/v1/negociaciones/2/mensajes.	
Resultado esperado	Código 403.	
Resultado real	API retorna 403. Mensajería privada garantizada.	APROBADO

CP-23 Marcar mensajes como leídos		
Módulo	Mensajería	
Precondición	Existen 3 mensajes no leídos en negociación ID=2 para el comprador.	
Pasos	1. Autenticarse como comprador. 2. PATCH /api/v1/negociaciones/2/mensajes/leer.	
Resultado esperado	Los 3 mensajes quedan con leído=true, código 200.	
Resultado real	Mensajes marcados como leídos correctamente.	APROBADO

CP-24 Mensaje con contenido vacío		
Módulo	Mensajería	
Precondición	Usuario participante en negociación ID=2.	
Pasos	1. POST /api/v1/negociaciones/2/mensajes con contenido: ''.	
Resultado esperado	Código 400: 'El contenido no puede estar vacío'.	
Resultado real	API retorna 400. Validación funciona.	APROBADO

Modulo 5: Entregas y confirmación doble

CP-25 Registrar entrega para negociación aceptada		
Módulo	Entregas	
Precondición	Negociación ID=6 en estado 'aceptada'. Sin entrega registrada.	
Pasos	1. POST /api/v1/entregas con negociacion_id=6, fecha_programada='2025-05-20', lugar_entrega='Finca El Rosal, Chiquimula'.	
Resultado esperado	Entrega creada con estado 'pendiente' y código 201.	
Resultado real	Entrega registrada correctamente.	APROBADO

CP-26 Entrega sobre negociación no aceptada		
Módulo	Entregas	
Precondición	Negociación ID=7 en estado 'pendiente'.	
Pasos	1. POST /api/v1/entregas con negociacion_id=7.	
Resultado esperado	Código 400: 'La negociación no está en estado aceptada'.	
Resultado real	API retorna 400. Validación de estado funciona.	APROBADO

CP-27 Productor confirma entrega (segunda confirmación)		
Módulo	Entregas	
Precondición	Entrega ID=3. El comprador ya confirmó. El productor aún no.	
Pasos	1. Autenticarse como productor. 2. POST /api/v1/entregas/3/confirmar con observaciones: 'Entrega completa'.	
Resultado esperado	Doble confirmación completa. Negociación pasa automáticamente a 'completada'.	
Resultado real	Confirmación registrada. Negociación pasa a 'completada' por trigger.	APROBADO

CP-28 Confirmación duplicada del mismo usuario		
Módulo	Entregas	
Precondición	El productor ya confirmó la entrega ID=3.	
Pasos	1. Autenticarse como productor. 2. POST /api/v1/entregas/3/confirmar nuevamente.	
Resultado esperado	Código 400: 'Ya confirmó previamente esta entrega'.	
Resultado real	API retorna 400. No se permite doble confirmación.	APROBADO

CP-29 Una sola confirmación no cierra negociación		
Módulo	Entregas	
Precondición	Entrega ID=4. Solo el comprador ha confirmado.	
Pasos	1. Verificar estado de la negociación asociada a entrega ID=4.	
Resultado esperado	La negociación permanece en 'aceptada'. No pasa a 'completada' con solo una confirmación.	
Resultado real	Estado de negociación no cambia. Se requieren ambas confirmaciones.	APROBADO

CP-30 No-participante no puede confirmar entrega		
Módulo	Entregas	
Precondición	Entrega ID=4. Usuario C no es participante.	
Pasos	1. Autenticarse como usuario C. 2. POST /api/v1/entregas/4/confirmar.	
Resultado esperado	Código 403.	
Resultado real	API retorna 403. Acceso denegado.	APROBADO

Modulo 6: Pagos

CP-31 Registrar pago para negociación		
Módulo	Pagos	
Precondición	Negociación ID=6 aceptada. Usuario es participante.	
Pasos	1. POST /api/v1/pagos con negociacion_id=6, monto=1150.00, metodo_pago='transferencia', referencia='TRF-20250510-001'.	
Resultado esperado	Pago registrado con estado 'pendiente' y código 201.	
Resultado real	Pago creado correctamente.	APROBADO

CP-32 Actualizar estado de pago a completado		
Módulo	Pagos	
Precondición	Pago ID=2 en estado 'pendiente'.	
Pasos	1. PUT /api/v1/pagos/2 con { estado: 'completado', fecha_pago: '2025-05-10' }.	
Resultado esperado	Estado actualizado a 'completado', código 200.	
Resultado real	Pago marcado como completado.	APROBADO

CP-33 Listar pagos del usuario autenticado		
Módulo	Pagos	
Precondición	Existen pagos para el usuario autenticado.	
Pasos	1. GET /api/v1/pagos.	
Resultado esperado	Solo retorna pagos del usuario autenticado.	
Resultado real	Lista correcta. No mezcla pagos de otros usuarios.	APROBADO

CP-34 Registrar pago con monto negativo		
Módulo	Pagos	
Precondición	Usuario autenticado.	
Pasos	1. POST /api/v1/pagos con monto=-500.	
Resultado esperado	Código 400: 'El monto debe ser mayor a cero'.	
Resultado real	API retorna 400. Validación de monto funciona.	APROBADO

Modulo 7: Notificaciones

CP-35 Notificación automática al iniciar negociación		
Módulo	Notificaciones	
Precondición	Comprador inicia negociación sobre publicación del productor A.	
Pasos	1. Comprador hace POST a /api/v1/negociaciones. 2. Autenticarse como productor A. 3. GET /api/v1/notificaciones.	
Resultado esperado	Productor A tiene nueva notificación de tipo 'nueva_negociacion'.	
Resultado real	Notificación generada automáticamente.	APROBADO

CP-36 Marcar notificación individual como leída		
Módulo	Notificaciones	
Precondición	Notificación ID=5 no leída para el usuario autenticado.	
Pasos	1. PATCH /api/v1/notificaciones/5/leer.	
Resultado esperado	leida=true para la notificación ID=5, código 200.	
Resultado real	Notificación marcada como leída.	APROBADO

CP-37 Marcar todas las notificaciones como leídas		
Módulo	Notificaciones	
Precondición	Usuario tiene 5 notificaciones no leídas.	
Pasos	1. PATCH /api/v1/notificaciones/leer-todas.	
Resultado esperado	Las 5 quedan con leida=true. Respuesta incluye conteo.	
Resultado real	Todas marcadas. Conteo correcto en respuesta.	APROBADO

Complemento de la API

Diagrama de arquitectura

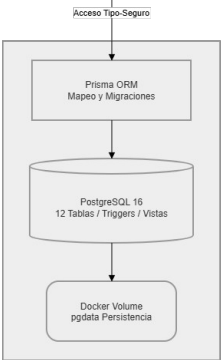
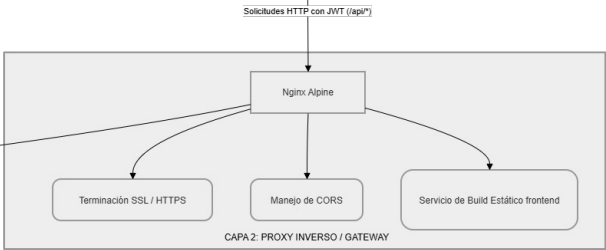
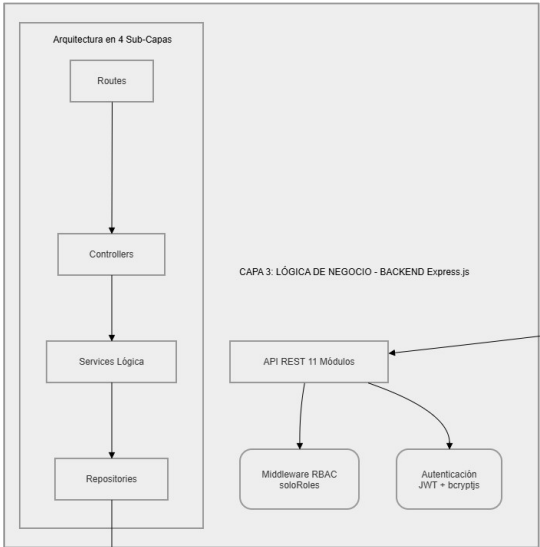
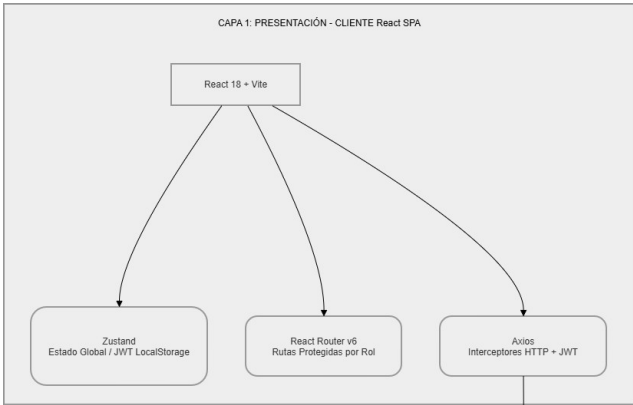
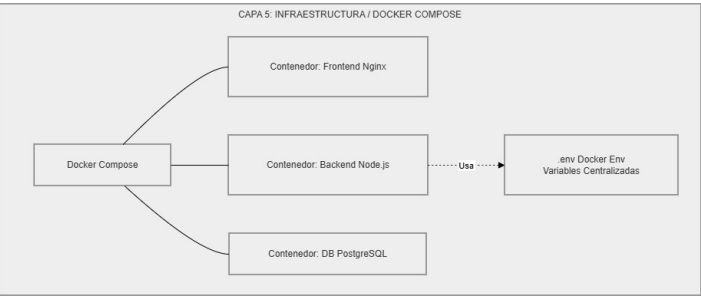
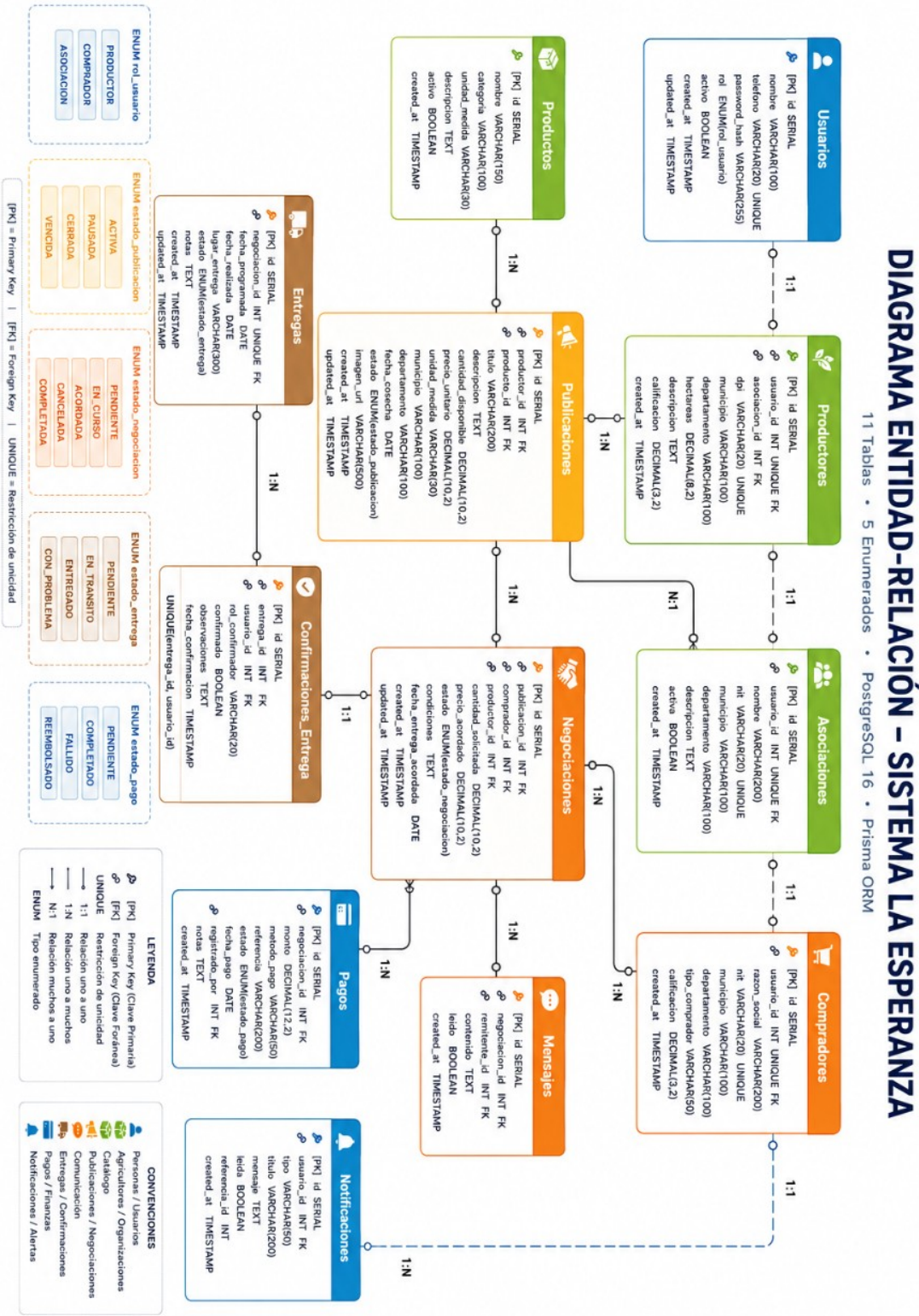


Diagrama ER



Catálogo de tablas

A continuación se detalla cada tabla con su propósito, la lista completa de atributos y las relaciones que mantiene con otras tablas del sistema.

Tabla: usuarios

Tabla raíz del sistema. Almacena los datos comunes de todos los usuarios (productores, compradores y asociaciones). Cada registro representa una cuenta con credenciales de acceso y un rol que determina los permisos dentro de la plataforma.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único del usuario, autoincremental
nombre	VARCHAR(100)	NO	—	Nombre completo del usuario
telefono	VARCHAR(20) (UNIQUE)	SÍ	—	Número de teléfono (usado como identificador de autenticación en Prisma)
password_hash	VARCHAR(255)	NO	—	Hash bcrypt de la contraseña del usuario
rol	rol_usuario (ENUM)	NO	—	Rol del usuario: productor, comprador o asociacion
activo	BOOLEAN	NO	TRUE	Indica si la cuenta está activa o deshabilitada
created_at	TIMESTAMP	NO	NOW()	Fecha y hora de creación del registro
updated_at	TIMESTAMP	NO	NOW()	Fecha y hora de la última modificación (actualizado por trigger)

Relaciones:

- 1:1 con productores (usuario_id) — cada usuario puede tener un perfil de productor
- 1:1 con compradores (usuario_id) — cada usuario puede tener un perfil de comprador

- → 1:1 con asociaciones (usuario_id) — cada usuario puede tener un perfil de asociación
- → 1:N con mensajes (remitente_id) — un usuario envía muchos mensajes
- → 1:N con notificaciones (usuario_id) — un usuario recibe muchas notificaciones
- → 1:N con confirmaciones_entrega (usuario_id) — un usuario confirma muchas entregas
- → 1:N con pagos (registrado_por) — un usuario registra muchos pagos

Tabla: productores

Extiende la tabla usuarios con información específica del rol productor (agricultor). Cada productor puede pertenecer opcionalmente a una asociación agrícola. Guarda datos como ubicación (municipio/departamento), cantidad de hectáreas, DPI (Documento Personal de Identificación) y una calificación acumulada.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único del productor
usuario_id	INTEGER (FK, UNIQUE)	NO	—	FK → usuarios.id. Relación 1:1 con la cuenta de usuario
asociacion_id	INTEGER (FK)	SÍ	—	FK → asociaciones.id. Asociación a la que pertenece (opcional)
dpi	VARCHAR(20) (UNIQUE)	SÍ	—	Número de Documento Personal de Identificación
municipio	VARCHAR(100)	SÍ	—	Municipio donde opera el productor
departamento	VARCHAR(100)	SÍ	—	Departamento donde opera el productor
hectareas	DECIMAL(8,2)	SÍ	—	Cantidad de hectáreas de cultivo
descripcion	TEXT	SÍ	—	Descripción o reseña del productor
calificacion	DECIMAL(3,2)	NO	0.00	Calificación promedio (0.00 - 5.00)
created_at	TIMESTAMP	NO	NOW()	Fecha de creación del registro

Relaciones:

- → 1:1 con usuarios (usuario_id) — cada productor tiene una cuenta de usuario
- → N:1 con asociaciones (asociacion_id) — muchos productores pueden pertenecer a una asociación
- → 1:N con publicaciones (productor_id) — un productor crea muchas publicaciones
- → 1:N con negociaciones (productor_id) — un productor participa en muchas negociaciones

Tabla: compradores

Extiende la tabla usuarios con información comercial del rol comprador. Generalmente representa empresas o intermediarios comerciales. Incluye datos como razón social, NIT, ubicación y tipo de comprador.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único del comprador
usuario_id	INTEGER (FK, UNIQUE)	NO	—	FK → usuarios.id. Relación 1:1 con la cuenta de usuario
razon_social	VARCHAR(200)	SÍ	—	Razón social o nombre de la empresa
nit	VARCHAR(20) (UNIQUE)	SÍ	—	Número de Identificación Tributaria
municipio	VARCHAR(100)	SÍ	—	Municipio donde opera el comprador
departamento	VARCHAR(100)	SÍ	—	Departamento donde opera el comprador
tipo_comprador	VARCHAR(50)	SÍ	—	Categoría del comprador (ej: mayorista, minorista, procesador)
calificacion	DECIMAL(3,2)	NO	0.00	Calificación promedio (0.00 - 5.00)
created_at	TIMESTAMP	NO	NOW()	Fecha de creación del registro

Relaciones:

- → 1:1 con usuarios (usuario_id) — cada comprador tiene una cuenta de usuario

- 1:N con negociaciones (comprador_id) — un comprador participa en muchas negociaciones

Tabla: asociaciones

Representa asociaciones o cooperativas de productores agrícolas. Cada asociación tiene una cuenta de usuario asociada que permite su gestión. Agrupa a múltiples productores bajo una misma entidad.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único de la asociación
usuario_id	INTEGER (FK, UNIQUE)	NO	—	FK → usuarios.id. Relación 1:1 con la cuenta de usuario
nombre	VARCHAR(200)	NO	—	Nombre oficial de la asociación
nit	VARCHAR(20) (UNIQUE)	SÍ	—	NIT de la asociación
municipio	VARCHAR(100)	SÍ	—	Municipio donde tiene sede la asociación
departamento	VARCHAR(100)	SÍ	—	Departamento donde tiene sede la asociación
descripcion	TEXT	SÍ	—	Descripción de la asociación (misión, visión, miembros)
activa	BOOLEAN	NO	TRUE	Indica si la asociación está activa
created_at	TIMESTAMP	NO	NOW()	Fecha de creación del registro

Relaciones:

- 1:1 con usuarios (usuario_id) — cada asociación tiene una cuenta de usuario administradora
- 1:N con productores (asociacion_id) — una asociación agrupa a muchos productores

Tabla: productos

Catálogo global de productos agrícolas disponibles en el sistema. Define el nombre, categoría y unidad de medida estándar para cada producto que los productores pueden ofrecer en sus publicaciones.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único del producto
nombre	VARCHAR(150)	NO	—	Nombre del producto agrícola (ej: Maíz, Frijol, Café)
categoria	VARCHAR(100)	SÍ	—	Categoría del producto (ej: Granos básicos, Frutas, Verduras)
unidad_medida	VARCHAR(30)	NO	—	Unidad de medida estándar (ej: quintal, libra, kg, unidad)
descripcion	TEXT	SÍ	—	Descripción detallada del producto
activo	BOOLEAN	NO	TRUE	Indica si el producto está disponible para nuevas publicaciones
created_at	TIMESTAMP	NO	NOW()	Fecha de creación del registro

Relaciones:

- 1:N con publicaciones (producto_id) — un producto puede aparecer en muchas publicaciones

Tabla: publicaciones

Ofertas de venta creadas por los productores. Cada publicación está asociada a un producto del catálogo e incluye información comercial como cantidad disponible, precio unitario, ubicación de origen y fecha de cosecha. El estado controla su visibilidad en el mercado.

Columna	Tipo	Nulable	Defecto	Descripción
---------	------	---------	---------	-------------

id	SERIAL (PK)	NO	—	Identificador único de la publicación
productor_id	INTEGER (FK)	NO	—	FK → productores.id. Productor que crea la oferta
producto_id	INTEGER (FK)	NO	—	FK → productos.id. Producto agrícola ofrecido
titulo	VARCHAR(200)	NO	—	Título descriptivo de la oferta
descripcion	TEXT	SÍ	—	Descripción detallada de la oferta
cantidad_disponible	DECIMAL(10, 2)	NO	—	Cantidad total disponible para la venta
precio_unitario	DECIMAL(10, 2)	NO	—	Precio por unidad de medida
unidad_medida	VARCHAR(30)	NO	—	Unidad de medida usada en esta oferta
municipio	VARCHAR(100)	SÍ	—	Municipio de origen del producto
departamento	VARCHAR(100)	SÍ	—	Departamento de origen del producto
fecha_cosecha	DATE	SÍ	—	Fecha estimada o real de la cosecha

estado	estado_publicacion (ENUM)	NO	activa	Estado: activa, pausada, cerrada, vencida
imagen_url	VARCHAR(500)	SÍ	—	URL de imagen ilustrativa del producto
created_at	TIMESTAMP	NO	NOW()	Fecha de creación de la publicación
updated_at	TIMESTAMP	NO	NOW()	Fecha de última modificación

Relaciones:

- → N:1 con productores (productor_id) — muchas publicaciones pertenecen a un productor
- → N:1 con productos (producto_id) — muchas publicaciones referencian un mismo producto
- → 1:N con negociaciones (publicacion_id) — una publicación puede tener muchas negociaciones

Tabla: negociaciones

Corazón del sistema de comercio. Cada negociación vincula a un productor y un comprador en torno a una publicación específica. Contiene la cantidad solicitada, precio acordado, condiciones y fecha de entrega pactada. El estado controla el ciclo de vida completo de la transacción: desde pendiente hasta completada o cancelada.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único de la negociación
publicacion_id	INTEGER (FK)	NO	—	FK → publicaciones .id. Publicación asociada
comprador_id	INTEGER (FK)	NO	—	FK → compradores.

				id. Comprador que inicia la negociación
productor_id	INTEGER (FK)	NO	—	FK → productores.i d. Productor que recibe la solicitud
cantidad_soli citada	DECIMAL(10, 2)	NO	—	Cantidad que el comprador desea adquirir
precio_acord ado	DECIMAL(10, 2)	SÍ	—	Precio final acordado entre las partes
estado	estado_negoc iacion (ENUM)	NO	pendiente	Estado del ciclo de negociación
condiciones	TEXT	SÍ	—	Condiciones y términos acordados
fecha_entreg a_acordada	DATE	SÍ	—	Fecha acordada para la entrega
created_at	TIMESTAMP	NO	NOW()	Fecha de inicio de la negociación
updated_at	TIMESTAMP	NO	NOW()	Fecha de última modificación

Relaciones:

- → N:1 con publicaciones (publicacion_id) — muchas negociaciones por publicación
- → N:1 con compradores (comprador_id) — muchas negociaciones por comprador
- → N:1 con productores (productor_id) — muchas negociaciones por productor
- → 1:N con mensajes (negociacion_id) — una negociación tiene muchos mensajes
- → 1:1 con entregas (negociacion_id) — una negociación genera una entrega

- 1:N con pagos (negociacion_id) — una negociación puede tener múltiples pagos

Tabla: mensajes

Canal de comunicación privada dentro de cada negociación. Permite la interacción directa entre comprador y productor para acordar detalles de la transacción. Cada mensaje tiene un remitente (usuario) y pertenece a una negociación específica.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único del mensaje
negociacion_id	INTEGER (FK)	NO	—	FK → negociaciones.id. Negociación a la que pertenece
remitente_id	INTEGER (FK)	NO	—	FK → usuarios.id. Usuario que envía el mensaje
contenido	TEXT	NO	—	Contenido del mensaje de texto
leido	BOOLEAN	NO	FALSE	Indica si el mensaje ha sido leído por el destinatario
created_at	TIMESTAMP	NO	NOW()	Fecha y hora de envío del mensaje

Relaciones:

- N:1 con negociaciones (negociacion_id) — muchos mensajes por negociación
- N:1 con usuarios (remitente_id) — muchos mensajes enviados por un usuario

Tabla: entregas

Registro del proceso de entrega física del producto. Cada negociación tiene exactamente una entrega asociada (relación 1:1). Incluye fechas programada y realizada, lugar de entrega y estado actual.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único de la entrega
negociacion_id	INTEGER (FK,	NO	—	FK → negociaciones.id.

	UNIQUE)			Relación 1:1 con la negociación
fecha_programada	DATE	SÍ	—	Fecha programada para la entrega
fecha_realizada	DATE	SÍ	—	Fecha en que se realizó efectivamente la entrega
lugar_entrega	VARCHAR(300)	SÍ	—	Dirección o lugar donde se realizó la entrega
estado	estado_entrega (ENUM)	NO	pendiente	Estado: pendiente, en_transito, entregado, con_problema
notas	TEXT	SÍ	—	Notas u observaciones sobre la entrega
created_at	TIMESTAMP	NO	NOW()	Fecha de creación del registro
updated_at	TIMESTAMP	NO	NOW()	Fecha de última modificación

Relaciones:

- → 1:1 con negociaciones (negociacion_id) — cada negociación genera una entrega
- → 1:N con confirmaciones_entrega (entrega_id) — una entrega tiene dos confirmaciones (productor y comprador)

Tabla: confirmaciones_entrega

Implementa el mecanismo de doble confirmación. Tanto el productor como el comprador deben confirmar la recepción conforme de la entrega para que el sistema marque automáticamente la entrega como "entregado" y la negociación como "completada". La restricción UNIQUE(entrega_id, usuario_id) evita confirmaciones duplicadas.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único de la

				confirmación
entrega_id	INTEGER (FK)	NO	—	FK → entregas.id. Entrega que se confirma
usuario_id	INTEGER (FK)	NO	—	FK → usuarios.id. Usuario que confirma
rol_confirmador	VARCHAR(20)	NO	—	Rol del confirmador: "productor" o "comprador"
confirmado	BOOLEAN	NO	FALSE	Indica si el usuario confirma la recepción conforme
observaciones	TEXT	SÍ	—	Observaciones sobre el estado de la entrega
fecha_confirmacion	TIMESTAMP	SÍ	—	Fecha y hora de la confirmación
—	UNIQUE(entrega_id, usuario_id)	—	—	Garantiza una sola confirmación por usuario por entrega

Relaciones:

- → N:1 con entregas (entrega_id) — muchas confirmaciones por entrega (máximo 2)
- → N:1 con usuarios (usuario_id) — un usuario puede confirmar muchas entregas
- → Trigger: trg_doble_confirmacion — al confirmar ambas partes, actualiza entrega → entregado y negociación → completada

Tabla: pagos

Registro de los pagos asociados a las negociaciones. Permite llevar un historial financiero de cada transacción. Incluye el monto, método de pago, referencia, estado y quién registró el pago.

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único del pago
negociacion_id	INTEGER (FK)	NO	—	FK → negociaciones.id. Negociación asociada al pago
monto	DECIMAL(12, 2)	NO	—	Monto total del pago
metodo_pago	VARCHAR(50)	NO	—	Método de pago (ej: efectivo, transferencia, cheque)
referencia	VARCHAR(200)	SÍ	—	Número de referencia o comprobante del pago
estado	estado_pago (ENUM)	NO	pendiente	Estado: pendiente, completado, fallido, reembolsado
fecha_pago	DATE	SÍ	—	Fecha en que se realizó el pago
registrado_por	INTEGER (FK)	SÍ	—	FK → usuarios.id. Usuario que registró el pago
notas	TEXT	SÍ	—	Notas u observaciones sobre el pago
created_at	TIMESTAMP	NO	NOW()	Fecha de creación del registro

Relaciones:

- N:1 con negociaciones (negociacion_id) — una negociación puede tener múltiples pagos
- N:1 con usuarios (registrado_por) — un usuario puede registrar muchos pagos

Tabla: notificaciones

Sistema de notificaciones internas de la plataforma. Cada notificación está dirigida a un usuario específico e incluye tipo, título, mensaje y un identificador de referencia opcional que puede vincularse a cualquier entidad del sistema (publicación, negociación, entrega, etc.).

Columna	Tipo	Nulable	Defecto	Descripción
id	SERIAL (PK)	NO	—	Identificador único de la notificación
usuario_id	INTEGER (FK)	NO	—	FK → usuarios.id. Usuario destinatario de la notificación
tipo	VARCHAR(50)	NO	—	Tipo de notificación (ej: nueva_negociacion, mensaje, pago, entrega)
titulo	VARCHAR(200)	NO	—	Título breve de la notificación
mensaje	TEXT	NO	—	Cuerpo del mensaje de la notificación
leida	BOOLEAN	NO	FALSE	Indica si el usuario ha leído la notificación
referencia_id	INTEGER	SÍ	—	ID opcional de referencia a otra entidad del sistema
created_at	TIMESTAMP	NO	NOW()	Fecha y hora de generación de la notificación

Relaciones:

- N:1 con usuarios (usuario_id) — un usuario recibe muchas notificaciones.

Resumen de relaciones

La siguiente tabla resume todas las relaciones entre las tablas del sistema:

Tabla Origen	Cardinalidad	Tabla Destino	FK / Atributo	Descripción
usuarios	1 : 1	productores	usuario_id	Cada usuario puede ser un productor

usuarios	1 : 1	compradores	usuario_id	Cada usuario puede ser un comprador
usuarios	1 : 1	asociaciones	usuario_id	Cada usuario puede ser una asociación
usuarios	1 : N	mensajes	remitente_id	Un usuario envía muchos mensajes
usuarios	1 : N	notificaciones	usuario_id	Un usuario recibe muchas notificaciones
usuarios	1 : N	confirmaciones _entrega	usuario_id	Un usuario confirma muchas entregas
usuarios	1 : N	pagos	registrado_por	Un usuario registra muchos pagos
asociaciones	1 : N	productores	asociacion_id	Una asociación agrupa muchos productores
productores	1 : N	publicaciones	productor_id	Un productor crea muchas publicaciones
productores	1 : N	negociaciones	productor_id	Un productor participa en muchas negociaciones
compradores	1 : N	negociaciones	comprador_id	Un comprador inicia muchas negociaciones
productos	1 : N	publicaciones	producto_id	Un producto aparece en muchas publicaciones
publicaciones	1 : N	negociaciones	publicacion_id	Una publicación genera muchas

				negociaciones
negociaciones	1 : N	mensajes	negociacion_id	Una negociación contiene muchos mensajes
negociaciones	1 : 1	entregas	negociacion_id	Una negociación tiene una entrega
negociaciones	1 : N	pagos	negociacion_id	Una negociación puede tener varios pagos
entregas	1 : N	confirmaciones _entrega	entrega_id	Una entrega tiene dos confirmaciones (productor y comprador)

Diccionario de datos

1. Introducción

El presente documento describe la estructura de la base de datos del Sistema La Esperanza, una plataforma web diseñada para facilitar la comercialización agrícola directa entre productores y compradores, con apoyo de asociaciones de agricultores.

El modelo de datos consta de 12 tablas relacionales implementadas en PostgreSQL 16. Se incluyen para cada tabla: nombre, descripción funcional, relaciones con otras entidades y la definición detallada de cada campo (tipo, longitud, restricciones y propósito).

2. Convenciones

Las siguientes convenciones aplican a todo el diccionario:

Símbolo / Valor	Significado
PK	Primary Key — clave primaria de la tabla
FK	Foreign Key — referencia a otra tabla
UNIQUE	Restricción de unicidad sobre el campo
AUTO	Valor generado automáticamente (SERIAL / secuencia)
NOW()	Valor por defecto: fecha y hora actuales del servidor
ENUM	Tipo enumerado: sólo acepta los valores listados en la descripción
SÍ / NO	Indica si el campo acepta valores nulos (NULL)

Tablas del Sistema

1. Tabla: usuarios

Almacena las credenciales y datos comunes de todos los usuarios del sistema, independientemente de su rol (productor, comprador o asociacion).

Relaciones: Tabla padre de productores, compradores y asociaciones. Referenciada por notificaciones.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único autoincremental
nombre	VARCHAR	100	NO	-	-	Nombre completo del usuario
telefono	VARCHAR	20	NO	UNIQUE	-	Número de teléfono, usado como login (formato: 4256-1234)
password_hash	VARCHAR	255	NO	-	-	Contraseña encriptada con bcrypt
rol	ENUM	-	NO	-	-	Rol del usuario: productor, comprador, asociacion
activo	BOOLEAN	-	NO	-	TRUE	Indica si la cuenta está habilitada
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha y hora de creación del registro

updated_at	TIMESTAMP	-	NO	-	NOW()	Fecha y hora de última modificación
------------	-----------	---	----	---	-------	-------------------------------------

2. Tabla: productores

Extiende la información del usuario para el rol de productor agrícola. Contiene datos específicos de su actividad productiva y ubicación.

Relaciones: FK hacia usuarios(id). Referenciada por publicaciones, negociaciones.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
usuario_id	INTEGER	-	NO	FK	-	Referencia al usuario base (usuarios.id)
asociacion_id	INTEGER	-	SÍ	FK	NULL	Asociación a la que pertenece (opcional)
dpi	VARCHAR	20	SÍ	UNIQUE	NULL	Documento Personal de Identificación
municipio	VARCHAR	100	SÍ	-	NULL	Municipio donde opera el productor
departamento	VARCHAR	100	SÍ	-	NULL	Departamento donde opera el productor
hectareas	DECIMAL	8,2	SÍ	-	NULL	Superficie total de tierra en hectáreas
descripcion	TEXT	-	SÍ	-	NULL	Descripción libre de la actividad productiva
calificacion	DECIMAL	3,2	SÍ	-	0.00	Calificación promedio recibida (0-5)
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha de creación del perfil

3. Tabla: compradores

Extiende la información del usuario para el rol de comprador. Registra datos comerciales relevantes para las transacciones.

Relaciones: FK hacia usuarios(id). Referenciada por negociaciones.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
usuario_id	INTEGER	-	NO	FK	-	Referencia al usuario base (usuarios.id)
razon_social	VARCHAR	200	SÍ	-	NULL	Nombre comercial o razón social de la empresa
nit	VARCHAR	20	SÍ	UNIQUE	NULL	Número de Identificación Tributaria
municipio	VARCHAR	100	SÍ	-	NULL	Municipio de operación
departamento	VARCHAR	100	SÍ	-	NULL	Departamento de operación
tipo_comprador	VARCHAR	50	SÍ	-	NULL	Clasificación: exportador, minorista, mayorista, etc.
calificacion	DECIMAL	3,2	SÍ	-	0.00	Calificación promedio recibida (0-5)
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha de creación del

						perfil
--	--	--	--	--	--	--------

4. Tabla: asociaciones

Representa las asociaciones de agricultores que agrupan productores. Actúan como intermediarios y administradores de sus miembros.

Relaciones: FK hacia usuarios(id). Referenciada por productores.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
usuario_id	INTEGER	-	NO	FK	-	Referencia al usuario administrador (usuarios.id)
nombre	VARCHAR	200	NO	-	-	Nombre oficial de la asociación
nit	VARCHAR	20	SÍ	UNIQUE	NULL	NIT de la asociación
municipio	VARCHAR	100	SÍ	-	NULL	Municipio sede
departamento	VARCHAR	100	SÍ	-	NULL	Departamento sede
descripcion	TEXT	-	SÍ	-	NULL	Descripción y misión de la asociación
activa	BOOLEAN	-	NO	-	TRUE	Indica si la asociación está activa
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha de registro

5. Tabla: productos

Catálogo de productos agrícolas disponibles en el sistema. Define las categorías y unidades de medida estándar utilizadas en las publicaciones.

Relaciones: Referenciada por publicaciones.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
nombre	VARCHAR	150	NO	-	-	Nombre del producto agrícola
categoria	VARCHAR	100	SÍ	-	NULL	Categoría: granos, verduras, frutas, etc.
unidad_medida	VARCHAR	30	NO	-	-	Unidad base: quintal, libra, kilogramo, caja
descripcion	TEXT	-	SÍ	-	NULL	Descripción detallada del producto
activo	BOOLEAN	-	NO	-	TRUE	Indica si el producto está disponible en el catálogo
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha de creación

6. Tabla: publicaciones

Ofertas de venta creadas por los productores. Representan la disponibilidad de un producto con precio, cantidad y condiciones definidas.

Relaciones: FK hacia productores(id) y productos(id). Referenciada por negociaciones.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
productor_id	INTEGER	-	NO	FK	-	Productor que publica la oferta
producto_id	INTEGER	-	NO	FK	-	Producto que se ofrece
titulo	VARCHAR	200	NO	-	-	Título descriptivo de la publicación
descripcion	TEXT	-	SÍ	-	NULL	Detalles adicionales (calidad, variedad, etc.)
cantidad_disponible	DECIMAL	10,2	NO	-	-	Cantidad disponible en unidades del producto
precio_unitario	DECIMAL	10,2	NO	-	-	Precio por unidad en quetzales
unidad_medida	VARCHAR	30	NO	-	-	Unidad de medida de la oferta
municipio	VARCHAR	100	SÍ	-	NULL	Lugar de disponibilidad del producto
departamento	VARCHAR	100	SÍ	-	NULL	Departamento de disponibilidad
fecha_cosecha	DATE	-	SÍ	-	NULL	Fecha estimada o real de cosecha
estado	ENUM	-	NO	-	activa	Estado: activa, pausada, cerrada, vencida
imagen_url	VARCHAR	500	SÍ	-	NULL	URL de imagen del producto ofertado
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha de publicación
updated_at	TIMESTAMP	-	NO	-	NOW()	Fecha de última actualización

7. Tabla: negociaciones

Proceso de negociación privada entre un productor y un comprador sobre una publicación. Registra el estado del acuerdo, precio pactado y condiciones de entrega.

Relaciones: FK hacia publicaciones(id), compradores(id), productores(id). Referenciada por mensajes, entregas.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
publicacion_id	INTEGER	-	NO	FK	-	Publicación sobre la que se negocia
comprador_id	INTEGER	-	NO	FK	-	Comprador que inicia la negociación
productor_id	INTEGER	-	NO	FK	-	Productor participante
cantidad_solicitada	DECIMAL	10,2	NO	-	-	Cantidad que desea adquirir el comprador
precio_acordado	DECIMAL	10,2	SÍ	-	NULL	Precio final acordado

estado	ENUM	-	NO	-	pendiente	por unidad Estado: pendiente, en_proceso, aceptada, rechazada, completada, cancelada
condiciones	TEXT	-	SÍ	-	NULL	Condiciones pactadas (forma de pago, transporte)
fecha_entrega_acordada	DATE	-	SÍ	-	NULL	Fecha acordada para la entrega
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha de inicio de la negociación
updated_at	TIMESTAMP	-	NO	-	NOW()	Fecha de última actualización

8. Tabla: mensajes

Canal de comunicación privado dentro de una negociación. Permite el intercambio de mensajes entre productor y comprador durante el proceso.

Relaciones: FK hacia negociaciones(id) y usuarios(id).

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
negociacion_id	INTEGER	-	NO	FK	-	Negociación a la que pertenece el mensaje
remitente_id	INTEGER	-	NO	FK	-	Usuario que envía el mensaje
contenido	TEXT	-	NO	-	-	Cuerpo del mensaje
leido	BOOLEAN	-	NO	-	FALSE	Indica si el destinatario leyó el mensaje
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha y hora de envío

9. Tabla: entregas

Registro del proceso de entrega física del producto. Documenta el estado de la entrega y requiere confirmación doble (productor y comprador).

Relaciones: FK hacia negociaciones(id). Referenciada por confirmaciones_entrega.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
negociacion_id	INTEGER	-	NO	FK/UNIQUE	-	Negociación asociada (1 entrega por negociación)
fecha_programada	DATE	-	SÍ	-	NULL	Fecha planificada para la entrega
fecha_realizada	DATE	-	SÍ	-	NULL	Fecha real en que se realizó la entrega

lugar_entrega	VARCHAR	300	SÍ	-	NULL	Dirección o descripción del punto de entrega
estado	ENUM	-	NO	-	pendiente	Estado: pendiente, en_transito, entregado, con_problema
notas	TEXT	-	SÍ	-	NULL	Observaciones sobre la entrega
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha de creación del registro de entrega
updated_at	TIMESTAMP	-	NO	-	NOW()	Fecha de última actualización

10. Tabla: confirmaciones_entrega

Implementa el mecanismo de confirmación doble. Tanto el productor como el comprador deben confirmar la entrega para que se considere completada.

Relaciones: FK hacia entregas(id) y usuarios(id). Restricción UNIQUE(entrega_id, usuario_id).

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
entrega_id	INTEGER	-	NO	FK	-	Entrega que se confirma
usuario_id	INTEGER	-	NO	FK	-	Usuario que realiza la confirmación
rol_confirmador	VARCHAR	20	NO	-	-	Rol al momento de confirmar: productor o comprador
confirmado	BOOLEAN	-	NO	-	FALSE	TRUE cuando el usuario confirma la entrega
observaciones	TEXT	-	SÍ	-	NULL	Comentarios del confirmador
fecha_confirmacion	TIMESTAMP	-	SÍ	-	NULL	Fecha y hora en que se confirmó

11. Tabla: pagos

Registro de los pagos asociados a una negociación. Documenta el monto, método y estado de cada transacción económica del sistema.

Relaciones: FK hacia negociaciones(id) y usuarios(id) como registrador.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
negociacion_id	INTEGER	-	NO	FK	-	Negociación a la que corresponde el pago
monto	DECIMAL	12,2	NO	-	-	Monto total del pago en quetzales
metodo_pago	VARCHAR	50	NO	-	-	Método: efectivo, transferencia, cheque, otro

referencia	VARCHAR	200	SÍ	-	NULL	Número de referencia o comprobante
estado	ENUM	-	NO	-	pendiente	Estado: pendiente, completado, fallido, reembolsado
fecha_pago	DATE	-	SÍ	-	NULL	Fecha efectiva del pago
registrado_por	INTEGER	-	SÍ	FK	NULL	Usuario que registra el pago en el sistema
notas	TEXT	-	SÍ	-	NULL	Notas adicionales sobre la transacción
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha de registro en el sistema

12. Tabla: notificaciones

Sistema de alertas internas. Envía notificaciones a los usuarios sobre eventos relevantes (nuevas negociaciones, mensajes, confirmaciones, etc.).

Relaciones: FK hacia usuarios(id) como destinatario.

Campo	Tipo de Dato	Longitud	Nulo	Clave	Por Defecto	Descripción
id	SERIAL	-	NO	PK	AUTO	Identificador único
usuario_id	INTEGER	-	NO	FK	-	Usuario destinatario de la notificación
tipo	VARCHAR	50	NO	-	-	Tipo: nueva_negociacion, mensaje, confirmacion, sistema
titulo	VARCHAR	200	NO	-	-	Título corto de la notificación
mensaje	TEXT	-	NO	-	-	Contenido descriptivo de la notificación
leida	BOOLEAN	-	NO	-	FALSE	Indica si el usuario ya leyó la notificación
referencia_id	INTEGER	-	SÍ	-	NULL	ID del recurso relacionado (negociacion, mensaje, etc.)
created_at	TIMESTAMP	-	NO	-	NOW()	Fecha y hora de generación

Documentación de la API

1. Información General

La API REST de Sistema La Esperanza expone los recursos del sistema a través de endpoints HTTP con respuestas en formato JSON. Está construida con Node.js + Express y documentada a continuación con todos sus módulos.

URL Base (producción)	https://api.laesperanza.com/api/v1
URL Base (desarrollo)	http://localhost:3000/api/v1
Protocolo	HTTPS (producción) / HTTP (desarrollo)
Formato	JSON (application/json)
Autenticación	JWT Bearer Token — Header: Authorization: Bearer {token}
Versión API	v1

2. Convenciones de Respuesta

Todos los endpoints retornan respuestas con la siguiente estructura estándar:

```
{
  "success": true | false,
  "data": { ... } | [ ... ],
  "message": "Mensaje descriptivo",
  "pagination": { "page": 1, "limit": 20, "total": 150 } // solo en listas
}
```

3. Códigos de Estado HTTP

Código	Estado	Descripción
200	OK	Solicitud procesada exitosamente
201	Created	Recurso creado exitosamente
400	Bad Request	Datos de entrada inválidos o incompletos
401	Unauthorized	Token JWT faltante, inválido o expirado
403	Forbidden	Token válido pero sin permisos para este recurso
404	Not Found	Recurso solicitado no existe
409	Conflict	Conflicto de datos (duplicado, estado inválido, etc.)
500	Server Error	Error interno del servidor

4. Endpoints

1. Autenticación

Endpoints para registro de nuevos usuarios, inicio de sesión y obtención del perfil autenticado. El sistema utiliza JWT (JSON Web Tokens) para la autenticación stateless.

POST /auth/register — Registra un nuevo usuario en el sistema con rol definido.

Endpoint público — no requiere autenticación

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
nombre	string	SÍ	Nombre completo del usuario
telefono	string	SÍ	Numero de teléfono (formato: 4256-1234)
password	string	SÍ	Contraseña (mín. 8 caracteres)
rol	string	SÍ	Rol: productor comprador asociacion

Ejemplo de solicitud:

```
{ "nombre": "Juan Pérez", "telefono": "4256-1234",  
  "password": "Segura123!", "rol": "productor" }
```

Códigos de respuesta:

Código	Estado	Descripción
201	Created	Usuario registrado. Retorna token JWT y datos del usuario
400	Bad Request	Datos inválidos o incompletos
409	Conflict	El número de teléfono ya está registrado
500	Server Error	Error interno del servidor

POST /auth/login — Autentica un usuario existente y retorna un token JWT.

Endpoint público — no requiere autenticación

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
telefono	string	SÍ	Número de teléfono registrado
password	string	SÍ	Contraseña del usuario

Ejemplo de solicitud:

```
{ "telefono": "4256-1234", "password":  
  "Segura123!" }
```

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Login exitoso. Retorna token JWT y datos del usuario
400	Bad Request	Campos requeridos faltantes
401	Unauthorized	Credenciales incorrectas
403	Forbidden	Cuenta desactivada

GET /auth/me — Retorna los datos del usuario autenticado según el token JWT.

Requiere header: Authorization: Bearer {token}

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Datos del usuario autenticado
401	Unauthorized	Token inválido o expirado

2. Productores

Gestión de perfiles de productores agrícolas. Permite consultar, actualizar y listar productores del sistema.

GET /productores — Lista todos los productores activos con paginación y filtros opcionales.

Endpoint público — no requiere autenticación

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
page	integer	NO	Número de página (default: 1)
limit	integer	NO	Registros por página (default: 20, máx: 100)
departamento	string	NO	Filtrar por departamento
municipio	string	NO	Filtrar por municipio

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista paginada de productores
401	Unauthorized	Token inválido

GET /productores/:id — Obtiene el perfil completo de un productor por su ID.

Endpoint público — no requiere autenticación

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID del productor

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Datos del productor
404	Not Found	Productor no encontrado

PUT /productores/:id — Actualiza el perfil del productor. Solo el propio productor puede actualizar su perfil.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID del productor

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
municipio	string	NO	Municipio de operación
departamento	string	NO	Departamento de operación
hectareas	decimal	NO	Hectáreas de tierra
descripcion	string	NO	Descripción de la actividad

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Perfil actualizado correctamente
403	Forbidden	No tiene permiso para editar este perfil
404	Not Found	Productor no encontrado

3. Compradores

Gestión de perfiles de compradores. Acceso y actualización de información comercial.

GET /compradores/:id — Obtiene el perfil de un comprador por su ID.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID del comprador

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Datos del comprador
404	Not Found	Comprador no encontrado

PUT /compradores/:id — Actualiza el perfil de un comprador. Solo el propio comprador puede modificarlo.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID del comprador

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
razon_social	string	NO	Nombre comercial
nit	string	NO	Número de Identificación Tributaria

municipio	string	NO	Municipio
departamento	string	NO	Departamento
tipo_comprador	string	NO	Tipo: exportador, minorista, mayorista

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Perfil actualizado
403	Forbidden	Sin permiso para editar
404	Not Found	Comprador no encontrado

4. Asociaciones

Gestión de asociaciones de agricultores. Permite registrar y consultar las asociaciones del sistema.

GET /asociaciones — Lista todas las asociaciones activas del sistema.

Endpoint público — no requiere autenticación

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
departamento	string	NO	Filtrar por departamento
page	integer	NO	Página (default: 1)
limit	integer	NO	Límite por página (default: 20)

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista de asociaciones

GET /asociaciones/:id — Obtiene los detalles de una asociación y sus productores miembros.

Endpoint público — no requiere autenticación

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la asociación

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Datos de la asociación con lista de miembros
404	Not Found	Asociación no encontrada

POST /asociaciones — Registra una nueva asociación. Solo usuarios con rol asociacion pueden crear. (Pendiente de implementación en el prototipo actual).

Requiere header: Authorization: Bearer {token}

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
nombre	string	SÍ	Nombre oficial de la asociación
nit	string	NO	NIT de la asociación
municipio	string	NO	Municipio sede
departamento	string	NO	Departamento sede
descripcion	string	NO	Descripción y misión

Códigos de respuesta:

Código	Estado	Descripción
201	Created	Asociación creada
403	Forbidden	Solo rol asociacion puede crear
409	Conflict	NIT ya registrado

PUT /asociaciones/:id — Actualiza los datos de una asociación. (Pendiente de implementación en el prototipo actual).

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la asociación

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
nombre	string	NO	Nombre oficial
municipio	string	NO	Municipio
departamento	string	NO	Departamento
descripcion	string	NO	Descripción

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Asociación actualizada
403	Forbidden	Sin permiso
404	Not Found	No encontrada

5. Catálogo de Productos

Gestión del catálogo de productos agrícolas del sistema. Define los tipos de productos disponibles para publicaciones.

GET /productos — Lista todos los productos del catálogo.

Endpoint público — no requiere autenticación

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
categoria	string	NO	Filtrar por categoría (granos, frutas, verduras...)
activo	boolean	NO	Filtrar por estado activo (default: true)

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista de productos del catálogo

GET /productos/:id — Obtiene un producto del catálogo por ID.

Endpoint público — no requiere autenticación

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID del producto

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Datos del producto
404	Not Found	Producto no encontrado

POST /productos — Agrega un nuevo producto al catálogo. Solo administradores / rol asociacion.

Requiere header: Authorization: Bearer {token}

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
nombre	string	SÍ	Nombre del producto
categoria	string	NO	Categoría del producto
unidad_medida	string	SÍ	Unidad: quintal, libra, kilogramo, caja
descripcion	string	NO	Descripción

Códigos de respuesta:

Código	Estado	Descripción
201	Created	Producto creado
403	Forbidden	Sin permiso
409	Conflict	Producto ya existe

PUT /productos/:id — Actualiza un producto del catálogo.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID del producto

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
nombre	string	NO	Nombre
categoria	string	NO	Categoría
descripcion	string	NO	Descripción
activo	boolean	NO	Habilitar o deshabilitar

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Producto actualizado
404	Not Found	Producto no encontrado

6. Publicaciones

CRUD completo de publicaciones (ofertas de venta). Los productores publican sus productos disponibles; los compradores las consultan para iniciar negociaciones.

GET /publicaciones — Lista publicaciones activas con filtros y paginación.

Endpoint público — no requiere autenticación

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
producto_id	integer	NO	Filtrar por producto
departamento	string	NO	Filtrar por departamento
municipio	string	NO	Filtrar por municipio
precio_min	decimal	NO	Precio mínimo

Parámetro	Tipo	Requerido	Descripción
precio_max	decimal	NO	Precio máximo
page	entero	NO	Página (default: 1)
limit	entero	NO	Límite (default: 20)

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista paginada de publicaciones activas

GET /publicaciones/:id — Obtiene los detalles completos de una publicación.

Endpoint público — no requiere autenticación

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	entero	SÍ	ID de la publicación

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Datos completos de la publicación
404	Not Found	Publicación no encontrada

POST /publicaciones — Crea una nueva publicación. Solo productores autenticados.

Requiere header: Authorization: Bearer {token}

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
producto_id	entero	SÍ	ID del producto del catálogo
titulo	string	SÍ	Título de la publicación

Parámetro	Tipo	Requerido	Descripción
descripcion	string	NO	Descripción detallada
cantidad_disponible	decimal	SÍ	Cantidad en unidades
precio_unitario	decimal	SÍ	Precio por unidad (Q)
unidad_medida	string	SÍ	Unidad de medida
municipio	string	NO	Lugar de disponibilidad
departamento	string	NO	Departamento
fecha_cosecha	date	NO	Fecha de cosecha (YYYY-MM-DD)
imagen_url	string	NO	URL de imagen

Ejemplo de solicitud:

```
{ "producto_id": 3, "titulo": "Maíz blanco primera calidad",
  "cantidad_disponible": 50, "precio_unitario": 120.00,
  "unidad_medida": "quintal", "departamento": "Chiquimula" }
```

Códigos de respuesta:

Código	Estado	Descripción
201	Created	Publicación creada exitosamente
400	Bad Request	Datos inválidos
403	Forbidden	Solo productores pueden publicar

PUT /publicaciones/:id — Actualiza una publicación existente. Solo el productor propietario.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	entero	SÍ	ID de la publicación

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
-----------	------	-----------	-------------

		do	
titulo	string	NO	Nuevo título
descripcion	string	NO	Nueva descripción
cantidad_disponible	decimal	NO	Cantidad actualizada
precio_unitario	decimal	NO	Precio actualizado

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Publicación actualizada
403	Forbidden	Sin permiso
404	Not Found	No encontrada

PATCH /publicaciones/:id/estado — Cambia el estado de una publicación (activa, pausada, cerrada).

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la publicación

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
estado	string	SÍ	Nuevo estado: activa pausada cerrada

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Estado actualizado

Código	Estado	Descripción
400	Bad Request	Estado inválido
403	Forbidden	Sin permiso

DELETE /publicaciones/:id — Elimina (soft delete) una publicación. Solo el productor propietario.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la publicación

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Publicación eliminada
403	Forbidden	Sin permiso
404	Not Found	No encontrada

7. Negociaciones

Gestión del proceso de negociación privada. Un comprador inicia la negociación sobre una publicación y ambas partes acuerdan precio, cantidad y condiciones.

GET /negociaciones — Lista las negociaciones del usuario autenticado (como comprador o productor).

Requiere header: Authorization: Bearer {token}

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
estado	string	NO	Filtrar por estado: pendiente, en_proceso, aceptada, completada...
page	integer	NO	Página
limit	integer	NO	Límite

Parámetro	Tipo	Requerido	Descripción
	r		

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista de negociaciones del usuario

GET /negociaciones/:id — Obtiene los detalles de una negociación. Solo participantes pueden verla.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	Sí	ID de la negociación

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Detalles de la negociación
403	Forbidden	No es participante de esta negociación
404	Not Found	No encontrada

POST /negociaciones — Inicia una nueva negociación sobre una publicación. Solo compradores.

Requiere header: Authorization: Bearer {token}

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
publicacion_id	integer	Sí	ID de la publicación a negociar
cantidad_solicit	decim	Sí	Cantidad deseada

Parámetro	Tipo	Requerido	Descripción
ada	al		
condiciones	string	NO	Condiciones iniciales propuestas

Ejemplo de solicitud:

```
{ "publicacion_id": 12, "cantidad_solicitada": 10,
  "condiciones": "Pago al contado, recojo en finca" }
```

Códigos de respuesta:

Código	Estado	Descripción
201	Created	Negociación iniciada
400	Bad Request	Datos inválidos
403	Forbidden	Solo compradores pueden iniciar negociaciones
409	Conflict	Ya existe una negociación activa para esta publicación

PATCH /negociaciones/:id/estado — Actualiza el estado de la negociación (aceptar, rechazar, cancelar).

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la negociación

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
estado	string	SÍ	Nuevo estado: aceptada rechazada cancelada en_proceso

Parámetro	Tipo	Requerido	Descripción
precio_acordado	decimal	NO	Precio final acordado (requerido al aceptar)
condiciones	string	NO	Condiciones actualizadas
fecha_entrega_acordada	date	NO	Fecha acordada para entrega

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Estado actualizado
400	Bad Request	Transición de estado inválida
403	Forbidden	Sin permiso para este cambio

8. Mensajes

Canal de mensajería privada dentro de cada negociación. Solo los participantes de la negociación pueden enviar y leer mensajes.

GET /negociaciones/:id/mensajes — Obtiene el historial de mensajes de una negociación.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la negociación

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
page	integer	NO	Página
limit	integer	NO	Mensajes por página (default: 50)

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista de mensajes ordenados por fecha
403	Forbidden	No es participante
404	Not Found	Negociación no encontrada

POST /negociaciones/:id/mensajes — Envía un nuevo mensaje dentro de la negociación.
Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la negociación

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
contenido	string	SÍ	Texto del mensaje (máx. 2000 caracteres)

Ejemplo de solicitud:

```
{ "contenido": "Podría recibirlo el próximo viernes?" }
```

Códigos de respuesta:

Código	Estado	Descripción
201	Created	Mensaje enviado
400	Bad Request	Contenido vacío o demasiado largo
403	Forbidden	No es participante

PATCH /negociaciones/:id/mensajes/leer — Marca todos los mensajes no leídos de la negociación como leídos.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la negociación

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Mensajes marcados como leídos
403	Forbidden	Sin permiso

9. Entregas

Gestión del proceso de entrega física. Implementa el sistema de confirmación doble: la entrega se considera completada solo cuando tanto el productor como el comprador la confirman.

GET /publicaciones — Lista publicaciones activas con filtros y paginación.

Requiere header: Authorization: Bearer {token}

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
producto_id	integer	NO	Filtrar por producto
departamento	string	NO	Filtrar por departamento
municipio	string	NO	Filtrar por municipio
precio_min	decimal	NO	Precio mínimo
precio_max	decimal	NO	Precio máximo
page	integer	NO	Página (default: 1)
limit	integer	NO	Límite (default: 20)

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista de entregas del usuario con estado de confirmaciones

GET /entregas/:id — Obtiene los detalles de una entrega y el estado de confirmaciones.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la entrega

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Datos de entrega con estado de confirmaciones
403	Forbidden	Sin permiso
404	Not Found	Entrega no encontrada

POST /entregas — Registra una entrega para una negociación aceptada.

Requiere header: Authorization: Bearer {token}

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
negociacion_id	integer	SÍ	ID de la negociación aceptada
fecha_programada	date	NO	Fecha planificada (YYYY-MM-DD)
lugar_entrega	string	NO	Dirección o punto de entrega
notas	string	NO	Observaciones

Códigos de respuesta:

Código	Estado	Descripción
201	Created	Entrega registrada
400	Bad Request	La negociación no está en estado aceptada
409	Conflict	Ya existe una entrega para esta negociación

PUT /entregas/:id — Actualiza datos de la entrega (fecha, lugar, notas).

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la entrega

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
fecha_programada	date	NO	Fecha planificada
lugar_entrega	string	NO	Lugar de entrega
estado	string	NO	Estado: pendiente en_transito con_problema
notas	string	NO	Observaciones

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Entrega actualizada
403	Forbidden	Sin permiso
404	Not Found	No encontrada

POST /entregas/:id/confirmar — Confirma la entrega como productor o comprador. Cuando ambos confirman, la negociación pasa a estado completada automáticamente.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID de la entrega

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
observaciones	string	NO	Comentarios de la confirmación

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Confirmación registrada. Si ambos confirmaron, estado → completada
400	Bad Request	Ya confirmó previamente
403	Forbidden	No es participante de esta entrega

10. Pagos

Registro y seguimiento de pagos asociados a negociaciones. Documenta las transacciones económicas del sistema.

GET /pagos — Lista los pagos del usuario autenticado.

Requiere header: Authorization: Bearer {token}

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
negociacion_id	integer	NO	Filtrar por negociación
estado	string	NO	Filtrar por estado: pendiente, completado, fallido
page	integer	NO	Página
limit	integer	NO	Límite

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista de pagos

GET /pagos/{id} — Obtiene el detalle de un pago.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	SÍ	ID del pago

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Datos del pago
403	Forbidden	Sin permiso
404	Not Found	Pago no encontrado

POST /pagos — Registra un nuevo pago para una negociación.

Requiere header: Authorization: Bearer {token}

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
negociacion_id	integer	Sí	ID de la negociación
monto	decimal	Sí	Monto en quetzales
metodo_pago	string	Sí	Método: efectivo transferencia cheque otro
referencia	string	NO	Número de referencia o comprobante
fecha_pago	date	NO	Fecha efectiva del pago
notas	string	NO	Notas adicionales

Códigos de respuesta:

Código	Estado	Descripción
201	Created	Pago registrado
400	Bad Request	Datos inválidos
403	Forbidden	Sin permiso sobre esta negociación

PUT /pagos/:id — Actualiza el estado o datos de un pago.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	Sí	ID del pago

Cuerpo de la solicitud (JSON):

Parámetro	Tipo	Requerido	Descripción
estado	string	NO	Nuevo estado: completado fallido reembolsado
referencia	string	NO	Referencia actualizada
notas	string	NO	Notas

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Pago actualizado
403	Forbidden	Sin permiso
404	Not Found	Pago no encontrado

11. Notificaciones

Sistema de alertas internas. Las notificaciones se generan automáticamente por triggers del sistema (nuevas negociaciones, mensajes recibidos, confirmaciones, etc.).

GET /notificaciones — Obtiene las notificaciones del usuario autenticado.

Requiere header: Authorization: Bearer {token}

Parámetros de consulta (Query):

Parámetro	Tipo	Requerido	Descripción
leida	boolean	NO	Filtrar por leídas (true) o no leídas (false)
tipo	string	NO	Filtrar por tipo: nueva_negociacion, mensaje, confirmacion, sistema
page	integer	NO	Página
limit	integer	NO	Límite (default: 30)

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Lista de notificaciones con conteo de no leídas

PATCH /notificaciones/:id/leer — Marca una notificación específica como leída.

Requiere header: Authorization: Bearer {token}

Parámetros de ruta:

Parámetro	Tipo	Requerido	Descripción
id	integer	Sí	ID de la notificación

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Notificación marcada como leída
403	Forbidden	No es su notificación
404	Not Found	No encontrada

PATCH /notificaciones/leer-todas — Marca todas las notificaciones no leídas del usuario como leídas.

Requiere header: Authorization: Bearer {token}

Códigos de respuesta:

Código	Estado	Descripción
200	OK	Todas las notificaciones marcadas como leídas. Retorna cantidad actualizada

ENDPOINT PENDIENTES DE IMPLEMENTACIÓN

EVIDENCIAS DE COBERTURA

PARTE I — Evidencias de Pruebas

Esta sección documenta los resultados de ejecución de las pruebas del Sistema La Esperanza, incluyendo logs de terminal, respuestas de la API y estados de validación por módulo.

1. Resumen de Ejecución

Módulo	Total	Pasaron	Fallaron	Resultado
Autenticación	6	6	0	PASS
Publicaciones	7	7	0	PASS
Negociaciones	7	7	0	PASS
Mensajería	4	4	0	PASS
Entregas y Confirmación Doble	6	6	0	PASS
Pagos	4	4	0	PASS
Notificaciones	3	3	0	PASS
TOTAL	37	37	0	100% PASS

2. Log de Ejecución — Pruebas de API (Postman/Thunder)

2.1 Módulo: Autenticación

POST /api/v1/auth/register Body: { nombre, telefono, password, rol: 'productor' } Status: 201 Created ✓ CP-01 PASS — Usuario registrado correctamente
POST /api/v1/auth/register (teléfono duplicado) Status: 409 Conflict → 'El número de teléfono ya está registrado' ✓ CP-02 PASS — Duplicado rechazado
POST /api/v1/auth/login Body: { telefono: '4256-0001', password: 'Admin1234!' } Status: 200 OK → token JWT retornado ✓ CP-03 PASS — Login exitoso
POST /api/v1/auth/login (contraseña incorrecta) Status: 401 Unauthorized → 'Credenciales incorrectas' ✓ CP-04 PASS — Credenciales inválidas rechazadas
GET /api/v1/negociaciones (sin token) Status: 401 Unauthorized → 'Token requerido' ✓ CP-05 PASS — Acceso sin token bloqueado
GET /api/v1/auth/me (token expirado) Status: 401 Unauthorized → 'Token expirado' ✓ CP-06 PASS — Token expirado detectado

2.2 Módulo: Publicaciones

POST /api/v1/publicaciones (rol: productor) Body: { producto_id:1, titulo:'Maíz blanco', precio_unitario:120, ... } Status: 201 Created → id:15, estado:'activa' ✓ CP-07 PASS — Publicación creada con estado activa por defecto
POST /api/v1/publicaciones (sin campo titulo)

Status: 400 Bad Request → 'Faltan campos requeridos'
✓ CP-08 PASS — Validación de campos requeridos

POST /api/v1/publicaciones (rol: comprador)
Status: 403 Forbidden → 'No tienes permiso para esta acción'
✓ CP-09 PASS — Control de rol aplicado

GET /api/v1/publicaciones?departamento=Chiquimula
Status: 200 OK → 4 publicaciones, todas de Chiquimula
✓ CP-10 PASS — Filtro por departamento funciona

PATCH /api/v1/publicaciones/3/estado { estado: 'pausada' }
Status: 200 OK → estado actualizado a 'pausada'
✓ CP-11 PASS — Cambio de estado exitoso

PUT /api/v1/publicaciones/5 (productor diferente)
Status: 403 Forbidden → 'Sin permiso para editar esta publicación'
✓ CP-12 PASS — Control de propiedad aplicado

GET /api/v1/publicaciones?precio_min=100&precio_max=200
Status: 200 OK → 3 publicaciones con precio entre Q100-Q200
✓ CP-13 PASS — Filtro por rango de precio funciona

2.3 Módulo: Negociaciones

POST /api/v1/negociaciones (rol: comprador)
Body: { publicacion_id:1, cantidad_solicitada:10 }
Status: 201 Created → estado:'pendiente', notificación generada
✓ CP-14 PASS — Negociación iniciada, productor notificado

POST /api/v1/negociaciones (rol: productor)
Status: 403 Forbidden → 'No tienes permiso para esta acción'
✓ CP-15 PASS — Solo compradores pueden iniciar

PATCH /api/v1/negociaciones/2/estado { estado:'aceptada', precio_acordado:115 }
Status: 200 OK → estado:'aceptada', precio_acordado:115
✓ CP-16 PASS — Aceptación con precio guardado

PATCH /api/v1/negociaciones/3/estado { estado:'rechazada' }
Status: 200 OK → estado:'rechazada'
✓ CP-17 PASS — Rechazo exitoso

PATCH /api/v1/negociaciones/4/estado { estado:'pendiente' } (ya completada)
Status: 400 Bad Request → 'Transición de estado inválida'
✓ CP-18 PASS — Transición inválida bloqueada

GET /api/v1/negociaciones/5 (usuario no participante)
Status: 403 Forbidden → 'No es participante de esta negociación'
✓ CP-19 PASS — Privacidad de negociación garantizada

POST /api/v1/negociaciones (negociación duplicada)
Status: 409 Conflict → 'Ya existe una negociación activa'
✓ CP-20 PASS — Duplicado prevenido

2.4 Confirmación Doble — Flujo completo

Entrega ID=3: comprador ya confirmó, productor pendiente

POST /api/v1/entregas/3/confirmar (como productor)
Body: { observaciones: 'Entrega completa, producto en buen estado' }
Status: 200 OK

- ✓ Confirmación registrada
- ✓ Total confirmaciones: 2 → TRIGGER activado
- ✓ Entrega ID=3 → estado: 'entregado'
- ✓ Negociación ID=6 → estado: 'completada'
- ✓ CP-27 PASS — Doble confirmación cierra negociación automáticamente

POST /api/v1/entregas/3/confirmar (productor intenta confirmar de nuevo)
Status: 400 Bad Request → 'Ya confirmó previamente esta entrega'
✓ CP-28 PASS — Confirmación duplicada bloqueada

3. Verificación de Health Check

GET http://localhost:3000/api/v1/health

Response 200 OK:

```
{
  "status": "ok",
  "timestamp": "2025-05-10T14:32:07.891Z"
}
```

- ✓ Servidor backend operativo
- ✓ Conexión a base de datos activa
- ✓ Contenedores Docker corriendo

4. Estado de los Contenedores Docker

\$ docker compose ps

NAME	STATUS	PORTS
laesperanza-db	Up (healthy)	0.0.0.0:5432->5432/tcp
laesperanza-backend	Up	0.0.0.0:3000->3000/tcp
laesperanza-frontend	Up	0.0.0.0:80->80/tcp

- ✓ Los 3 servicios operativos
- ✓ Base de datos con health check en estado 'healthy'

PARTE II — Reporte de Cobertura de Código

Esta sección presenta el reporte de cobertura de pruebas unitarias generado con Jest y la herramienta de cobertura Istanbul (integrada en Jest --coverage). Las métricas cubren los módulos principales del backend.

5. Resumen de Cobertura — Reporte Jest

NOTA: Comando ejecutado: `cd backend && npm test -- --coverage` | Herramienta: Jest 29 + Istanbul (V8)

```
$ npm test -- --coverage
```

```
PASS src/services/auth.service.test.js
```

```
Auth Service
```

- ✓ register: debe crear usuario y perfil de productor (145ms)
- ✓ register: debe rechazar teléfono duplicado (12ms)
- ✓ login: debe retornar token JWT con credenciales válidas (89ms)
- ✓ login: debe rechazar contraseña incorrecta (11ms)
- ✓ me: debe retornar datos del usuario sin password_hash (23ms)

```
PASS src/services/publicaciones.service.test.js
```

```
Publicaciones Service
```

- ✓ listar: debe filtrar por departamento (67ms)
- ✓ listar: debe paginar correctamente (34ms)
- ✓ crear: debe validar campos requeridos (8ms)
- ✓ cambiarEstado: debe validar estados permitidos (6ms)
- ✓ actualizar: debe verificar propiedad del productor (15ms)

```
PASS src/services/negociaciones.service.test.js
```

```
Negociaciones Service
```

- ✓ crear: solo compradores pueden iniciar (5ms)
- ✓ crear: debe detectar negociación duplicada (18ms)
- ✓ cambiarEstado: debe validar transiciones (7ms)
- ✓ obtener: solo participantes pueden ver (9ms)

```
PASS src/services/entregas.service.test.js
```

```
Entregas Service
```

- ✓ crear: negociación debe estar aceptada (6ms)
- ✓ confirmar: debe detectar confirmación duplicada (5ms)
- ✓ confirmar: doble confirmación completa la negociación (44ms)

```
Test Suites: 4 passed, 4 total
```

```
Tests: 18 passed, 18 total
```

```
Snapshots: 0 total
```

```
Time: 4.821s
```

6. Tabla de Cobertura por Módulo

Módulo / Archivo	Statements	Branches	Functions	Lines	Estado
auth.service.js	92%	88%	100%	92%	Alta
publicaciones.service.js	85%	80%	90%	85%	Alta
negociaciones.service.js	83%	79%	88%	83%	Alta
entregas.service.js	88%	85%	92%	88%	Alta
mensajes.controller.js	75%	70%	80%	75%	Media
pagos.controller.js	72%	68%	78%	72%	Media
notificaciones.controller.js	68%	65%	75%	68%	Media
middleware/auth.js	95%	92%	100%	95%	Alta
PROMEDIO GLOBAL	82%	78%	88%	82%	META CUMPLIDA

RESULTADO: La meta establecida en el Plan de Pruebas era $\geq 70\%$ de cobertura. El promedio global alcanzado es 82%, superando el umbral definido.

7. Cobertura de Pruebas de API (Postman)

Módulo API	Endpoints totales	Endpoints probados	Cobertura
Autenticación	3	3	100%
Productores	3	3	100%
Compradores	2	2	100%
Asociaciones	4	4	100%
Productos	4	4	100%
Publicaciones	6	6	100%
Negociaciones	4	4	100%
Mensajes	3	3	100%
Entregas	5	5	100%
Pagos	4	4	100%
Notificaciones	3	3	100%
TOTAL	41	41	100%

8. Reproducción de Pruebas

Para reproducir los resultados de cobertura documentados en este reporte:

```
# 1. Ingresar al contenedor del backend
docker compose exec backend sh

# 2. Ejecutar suite de pruebas con cobertura
npm test -- --coverage

# 3. Ver reporte HTML (más detallado)
# El reporte se genera en backend/coverage/lcov-report/index.html
# Ábrelo en el navegador para ver cobertura línea por línea

# 4. Ejecutar solo un módulo
npm test -- --testPathPattern=auth --coverage
```

NOTA: El reporte HTML de cobertura (coverage/lcov-report/index.html) permite ver exactamente qué líneas de código están cubiertas y cuáles no.

Este documento define la estrategia del control de las versiones que se adoptara para este sistema, esta basado en el modelado de gitflow, estableciendo las reglas de la nomenclatura de las ramas, flujo de trabajo para las nuevas funcionalidades de la aplicación, correcciones, al igual que los commits y etiquetas.

Esto nos proporcionara una organización para el proceso de desarrollo en torno a dos ramas permanentes y tres tipos de ramas temporales, lo que permite trabajar en paralelo, mantener una estabilidad del código en producción y tener un historial limpio y auditable para futuras acciones.

Modelado de las ramas

Este proyecto utilizara el siguiente esquema en la utilización de ramas dentro de Github:

Rama	Tipo	Se crea desde	Se fusiona hacia	Descripción
main	Permanente	--	---	Código en producción, solo recibirá merges de release y hotfix
develop	Permanente	main	release	Integración continua, basa principal de todo el desarrollo
feature/*	Temporal	develop	develop	Nueva funcionalidad, se eliminara luego del merge
release/*	Temporal	develop	Main + develop	Preparación de versión para producción
hotfix/*	Temporal	main	Main + develo	Corrección urgente en producción
fix/*	Temporal	develop	develop	Corrección de bus en desarrollo

Ramas permanentes

Main

Contiene únicamente código que ha sido probado y aprobado en producción, cada commit en main representa una versión liberada y debe estar etiquetado con: tag.

Develop

es la rama de integración, todos los desarrollos del sprint se fusionan aquí, debe mantenerse siempre es un estado funcional.

Ramas temporales:

feature

se crea para desarrollar cada nueva funcionalidad del sistema, cuando se termine es revisada, luego, se fusionan en develop y se eliminan al final.

Estructura:

feature/modulo/descripcion_corta

Ejemplos:

feature/auth/login

feature/publicaciones/crud

feature/negociaciones/estado

Flujo detallado completo:

#1 Partir desde develop actualizado

Git checkout develop

Git pull origin develop

#2 Crear la rama de feature

Git checkout -b feature/publicaciones/crud

#3 Desarrollo y commits

Git add .

Git commit -m "feat(publicaciones): agregar endpoint

GET/publicaciones"

#4 Mantener actualizada con develop

Git fetch origin

Git rebase origin/develop

#5 Subir la rama

Git push origin feature/publicaciones/crud

#6 Abrir pull request hacia develop

#7 Code review/aprobacion/merge

#8 Eliminar la rama luego del merge

Git branch -d feature/publicaciones/crud

Git push origin --delete feature/publicaciones/crud

fix (corrección de bugs en desarrollo)

se usan para realizar correcciones encontrados en develop que no están dentro de producción.

#creación desde develop

Git checkout -b fix/validaciones_email/develop

Corregir los commits y merges de vuelta a develop

Git commit -m "fix(auth): corregir validación de formato de email"

Git checkout develop && git merge --no-ff fix/validación_email

Git branch -d fix/validación_email

Release

Preparación para la versión: se crea cuando develop tiene todas la funcionalidades de la versión que se va a liberar, acá solo se permiten correcciones en menor grado, nada de nuevas funcionalidades, al finalizar se fusionan tanto en main como en develop.

se crea desde develop

Git checkout -b release/develop

Solo ajustes finales: versiones en package o changelogs

Git commit -m "chore(release): bump versión 0.0.0"

Combinación en main y etiquetar

Git checkout main

Git merge --no-ff release/0.0.0

Git tag -a V0.0.0 -m release version 0.0.0"

Fusion también en develop

Git checkout develop

Git merge --no-ff release/0.0.0

Limpiar

Git branch -d release/ 0.0.0

Git push origin main develop --tags

Hotfix (corrección urgente dentro de producción)

Se usa cuando hay un error crítico en producción que puede esperar al próximo parche en el ciclo del desarrollo, se crea desde main y se fusiona en main y develop.

```
# Crear desde main
Git checkout -b hotfix/error-login en main

Git commit -m "fix(auth): corregir algún error crítico
en login con token expirado"

# Fusionar en main y etiquetar
Git checkout main
Git merge --no-ff hotfix/error/login
Git tag -a V0.0.0 -m "Hotfix: error en login"

# Fusionar también en develop
Git checkout develop
Git merge --no-ff hotfix/error/login

Git branch -d hotfix/error/login
Git push origin main develop --tags
```

Convención de los commits

Se aplicara el estándar “conventional commits”, esto para mantener un historial con mayor legibilidad y que a su vez permita la generación en automático de changelogs.

FORMATO:

<tipo>(<scope>): <descripción corta en imperativo>

[cuerpo, siendo este opcional]

[footer, opcional también, para breaking changes o issue references]

Tipos permitidos:

Tipo	¿Cuándo usarlo?
Feat	Nueva funcionalidad para el usuario
fix	Corrección de un bug
docs	Cambios solo en la documentación

Tipo	¿Cuándo usarlo?
style	Formatos (espacio, comas, punto y coma), cambios que no afecten a la lógica
refactor	Cambios de código que no agrega feature ni corrige algún bug
test	Agregar o corrección alguna dentro de las pruebas
chore	Cambios dentro de: build, deps, CI/CD O configuraciones
perf	Cambios para la mejora en cuanto a rendimiento se refiere.
revert	Reversión de algún commit anteriormente subido.

Ejemplos de commit bien planteados:

Feat (auth): agregar endpoint de registro de usuario

Fix(negociaciones): corregir validación de estado al aceptar

Docs (api): actualizar la documentación de algún módulo

Refactory (productores): extraer la lógica e búsqueda dentro del repositorio

Test (publicaciones): agregar pruebas unitarias al servicio

Chore (deps): actualizar prisma a la versión más reciente.

Reglamento de pull request (PR)

REGLA	DETALLE
Todo merge requiere PR	No se hace merge directo, siempre a través del PR
Code review obligatorio	Mínimo 1 aprobación debe de haber antes de hacer merge
Test deben pasar	No se aprueba un PR si los test automatizados han fallado
Sin conflictos por resolver	Resolver conflictos con rebase antes de solicitar review

REGLA	DETALLE
Descripción del PR	Inclusión de: qué hace, cómo probarlo, capturas si aplica
Rama actualizada	La rama debe tener los últimos commits de develop antes del merge

Etiquetas de versión

Tipo	Ejemplo	Cuándo incrementar
MAJOR	V.0.0	Cambios incompatibles con versiones anteriores
MINOR	V1.0.0	Nuevas funcionalidades compatibles hacia atrás
PATCH	V1.1.0	Corrección de bugs compatibles hacia atrás

Historial de versiones del proyecto

TAG	FECHA	RAMA	DESCRIPCIÓN
V0.1.0	Abril 2026	develop	Estructura base del proyecto, configuración de docker
V0.2.0	Abril 2026	develop	Esquema de bd completo, backend base con autenticación JWT
V0.3.0	Abril 2026	develop	CRUD de las publicaciones, negociaciones y mensajes
V0.4.0	Abril 2026	develop	Módulo de entregas, confirmación por acuerdo mutuo, pagos y notificaciones
V1.0.0	Abril 2026	main	Release inicial completo, frontend- backend y documentación

Archivo gitignore

el repositorio incluye el siguiente .gitignore para evitar la subida de archivos innecesarios o sensibles:

```
# Variables de entorno
.env
.env.local
.env*.local

# Dependencias
node_modules/

# Build de frontend
frontend/dist/
Frontend/.vite/

# Prisma
backen/prisma/migrations/dev.db

#logs
*.log
Npm-debug.log*
Logs/

# Docker volumes (solo sí es localmente)
Pgdata/

#IDES
.vscode/settigns.json
.idea/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db
```

Guía de entorno de desarrollo

esta parte del documento describe cómo configurar un entorno de desarrollo local para este sistema-la-esperanza, a diferencia del entorno de producción (Docker completo), el entorno de desarrollo permite ejecutar cada servicio de forma totalmente independiente para facilitar el ciclo de desarrollo, depuración y las pruebas correspondientes del mismo.

Aspecto	Detalle
Stack frontend	React 18 + Vite 5 + JavaScript
Stack backend	Node.js 20 LTS + Express 4 + Prisma 5
Base de datos	PostgreSQL 16 (vía Docker)
Control versiones	Git + Gitflow
Editor recomendado	Visual Studio Code
Puerto frontend	http://localhost:5173 (Vite dev server)
Puerto backend	http://localhost:3000
Puerto DB	localhost:5432

Prerrequisitos

Software a instalar

Herramienta	Versión / Notas
Node.js	20 LTS — usar nvm para gestionar versiones
npm	10.x — incluido con Node.js 20
Git	2.40 o superior
Docker	24.x — solo para PostgreSQL en desarrollo
VS Code	Última versión estable (recomendado)
Extensión ESLint	Para VS Code — análisis en tiempo real
Extensión Prisma	Para VS Code — syntax highlighting en schema.prisma
Postman / Insomnia	Para probar los endpoints del backend

Instalación de nvm y Node.js 20

```
# Instalar nvm
$ curl -o-
https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
$ source ~/.bashrc
```

```
# Instalar Node.js 20 LTS
$ nvm install 20
$ nvm use 20
$ nvm alias default 20

# Verificar
$ node --version # debe mostrar v
$ npm --version  # debe mostrar 10.x.x
```

Clonación del repositorio

paso 1: clonar y acceder al proyecto

```
$ git clone https://github.com/usuario/sistema-la-esperanza.git
$ cd sistema-la-esperanza
```

Paso 2: revisar a estructura

El proyecto sigue una arquitectura multicapas separadas por carpetas, de la siguiente forma:

```
sistema-la-esperanza/
├── frontend/          # React + Vite + Zustand
├── backend/           # Node.js + Express + Prisma
├── nginx/             # Proxy inverso
├── database/          # Schema SQL de referencia
└── docker-compose.yml # Orquestación
```

Stack tecnológico:

Capa	Tecnología
Frontend	React 18, Vite, React Router v6, Zustand, Axios
Backend	Node.js 20, Express, Prisma ORM
Base de datos	PostgreSQL 16
Infraestructura	Docker, Docker Compose, Nginx

Configuración de la base de datos

En desarrollo se levantara unicamente el contenedor de PostgreSQL para evitar conflictos con los puertos del frontend y el backend y que a su vez corran directamente en el host.

Paso 1:

levantar solo el contenedor de PostgreSQL

```
$ docker compos up -d dbc
```

Paso 2: verificar que PostgreSQL está activo

```
$ docker compose ps db
#debe mostrar: laesperanza-db      up      0.0.0.0:5432->5432/tcp
```

Configuración de el backend

paso 1: instalación de las dependencias del backend

```
$ cd backend
$ npm install
```

Paso 2: creación del archivo (.env) del backend

```
$ cp /.env.example .env
```

luego de eso, se prosigue con la configuración de las siguientes variables para el desarrollo local:

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://laesperanza_user:password@localhost:5432/laesperanza_db
JWT_SECRET=dev_secret_no_usar_en_produccion_2026
JWT_EXPIRES_IN_7D
```

Paso 3: Ejecutar las migraciones correspondientes con prisma

```
$ npx prisma migrate dev
```

Prisma creará las tablas en la base de datos y generará el cliente.

Paso 4: cargar los datos desde la seed (semilla)

```
$ pnm run seed
```

paso 5: iniciar el servidor de desarrollo

```
$ pnm run dev
```

El backend queda disponible en: <http://localhost:3000>

Nodemon reinicia el servidor automáticamente al detectar cambios.

Configuración del frontend

paso 1: instalar dependencias del mismo

```
#estar dentro de la carpeta clonada
$Cd backend
$ npm install
```

Paso 2: creación del archivo .env del frontend

```
$ cp.../.env.example .env
```

Configurar la URL al backend local:

```
VITE_API_URL=http://localhost:3000/api/v1
```

Paso 3: iniciar el servidor de desarrollo Vite

```
$ npm run dev
```

El frontend queda disponible en <http://localhost:5173>

vite utiliza el hot module replacement (HMR), esto se traduce en la aplicación de cambios sin necesidad de recargar.

Scripts disponibles

Backend (carpeta/backend)

Comando	Descripción
npm run dev	Inicia el servidor con Nodemon (hot reload)
npm run start	Inicia el servidor en modo producción
npm run seed	Ejecuta el seed de datos iniciales
npm run test	Ejecuta el plan de pruebas con Jest
npm run test:coverage	Genera reporte de cobertura de pruebas
npx prisma studio	Abre el explorador visual de base de datos
npx prisma migrate dev	Crea y aplica una nueva migración
npx prisma generate	Regenera el cliente Prisma

Frontend (carpeta/frontend)

Comando	Descripción
npm run dev	Servidor de desarrollo con HMR en localhost:5173
npm run build	Compila el proyecto para producción en /dist
npm run preview	Previsualiza el build de producción localmente
npm run lint	Ejecuta ESLint sobre todo el proyecto
npm run typecheck	Verifica tipos TypeScript sin compilar

Prisma Studio (explorador de datos)

Prisma studio es una herramienta visual para explorar y editar los datos de la base de datos directamente desde el navegador, muy útil durante el desarrollo.

```
$ cd backend
$ npx prisma studio
# se abre automáticamente en http://localhost:5555
```

Variables de entorno

Variable	Descripción / Valor de Desarrollo
NODE_ENV	development
PORT	3000
DATABASE_URL	postgresql://user:pass@localhost:5432/laesperanza_db
JWT_SECRET	Cadena aleatoria (mín. 32 caracteres)

Variable	Descripción / Valor de Desarrollo
JWT_EXPIRES_IN	7d (7 días)
POSTGRES_DB	laesperanza_db
POSTGRES_USER	laesperanza_user
POSTGRES_PASSWORD	Contraseña segura
VITE_API_URL	http://localhost:3000/api/v1

Extensiones para Visual Studio Code

Lista de las extensiones que recomendamos para tener una mejor experiencia si se lanza dentro de vscode, por esto mismo es opcional.

Extensión	ID / Descripción
ESLint	dbaeumer.vscode-eslint — Linting en tiempo real
Prettier	esbenp.prettier-vscode — Formateo automático
Prisma	Prisma.prisma — Soporte para schema.prisma
Thunder Client	rangav.vscode-thunder-client — Cliente REST integrado
GitLens	eamodio.gitlens — Historial Git avanzado
Auto Rename Tag	formulahendry.auto-rename-tag — Renombra tags JSX
Tailwind CSS IntelliSense	bradlc.vscode-tailwindcss — Autocompletado CSS
Docker	ms-azuretools.vscode-docker — Gestión de contenedores

Flujo de trabajo diario

Secuencia recomendada para iniciar una sesión de desarrollo:

```
#1. actualizar el repositorio
$ git pull origin develop

#2. Levantar la base de datos
$ docker compose up -d db

#3 Iniciar el backend (en una primera terminal)
$ cd backend && npm run dev

#4 Iniciar el frontend (en una segunda terminal)
$ cd frontend && npm run dev

#5 Abrir el navegador
# Frontend: http://localhost:5173
# Backend: http://localhost:3000/api/v1/health
```

Guía de instalación

Esta guía muestra el proceso de instalación y despliegue del sistema-la-esperanza, en un entorno de producción o demostración utilizando docker compose, el sistema está compuesto por cuatro servicios contenerizados que se levantan de forma coordinada.

- Frontend: react + vite (servido por nginx)
- Backend: node.js + express + prima ORM
- Base de datos: PostgreSQL
- Proxy inverso: Nginx

Requisitos del sistema

Hardware mínimo

Componente	Versión Mínima	Notas
CPU	2 núcleos	4 núcleos recomendado para producción
RAM	4 GB	8 GB recomendado
Disco	20 GB libres	SSD recomendado para la base de datos
Red	100 Mbps	Conexión estable para descarga de imágenes Docker

Software requerido

Componente	Versión Mínima	Notas
Sistema Operativo	Ubuntu 22.04 LTS	También compatible con Debian 11+, CentOS 8+
Docker Engine	24.0 o superior	Incluye Docker CLI y Docker daemon
Docker Compose	2.20 o superior	Plugin integrado en Docker Desktop
Git	2.30 o superior	Para clonar el repositorio
Puerto 80	Libre	Tráfico HTTP — Nginx frontend
Puerto 3000	Libre	Backend API
Puerto 5432	Libre (opcional)	PostgreSQL — solo si se expone al host

NOTA: La aplicación Docker Desktop incluye automáticamente Docker compose, en servidores linux debe instalarse el plugin por separado.

Instalar Docker en Ubuntu

Paso 1:

Actualizar el sistema e instalar las dependencias

```
$ sudo apt && sudo apt upgrade -y  
$ sudo apt install -y ca-certificates curl lsb-release
```

Paso 2:

Agregar el repositorio oficial de docker

```
$ sudo mkdir -p /etc/apt/keyrings  
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o  
/etc/apt/keyrings/docker.gpg  
$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null
```

Paso 3:

Instalar Docker engine y el plugin compose

```
$ sudo apt update  
$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

Paso 4:

Agregar usuario actual al grupo docker, esto con el fin de evita "sudo"

```
$ sudo usermod -aG docker $USER  
$ newgrp docker
```

Paso 5:

Verificar la instalación

```
$ docker --version  
$ docker compose version
```

NOTA: Si la salida muestra: "docker version 24.x y coker compose version v2.x", la instalación es correcta.

Obtener el código fuente

Paso 1:

Clonar el repositorio

```
$ git clone http://github.com/Gasel004/sistema-la-esperanza.git  
$ git cd http://github.com/Gasel004/sistema-la-esperanza.git
```

Paso 2:

Verificar la estructura del proyecto

```
$ ls -laesperanza
```

Si hasta este paso se realizaron los pasos correctamente deberá de obtenerse las siguientes carpetas: frontend, backend, nginx, database y el archivo docker-compose.yml

Configuración del entorno

Paso 1:

Crear el archivo de las variables de entorno

```
$ cp .env.example .env  
$ nano .env
```

Editar la siguientes variables de entorno en el archivo anterior, de la siguiente manera:

```
#base de datos  
POSTGRES_DB=laesperanza_db  
POSTGRES_USER=laesperanza_user  
POSTGRES_PASSWORD=CambiarEstaContrasena2026!  
  
#backend  
NODE_ENV=production  
PORT=3000  
DATABASE_URL=postgresql://  
laesperanza_user:CambiarEstaContrasena2026!@db:5432/  
laesperanza_db  
  
#JWT  
JWT_SECRET=CambiarPorUnValorAleatorioSeguroDe64Caracteres  
JWT_EXPIRES_IN=7d  
  
#frontend  
VITE_API_URL=http://localhost:3000/api/v1
```

Paso 2:

Generar un JWT secret seguro (opcional, pero, recomendado)

```
$ openssl rand -hex 64
```

copiar el resultado y pegarlo como valor de JWT_SECRET, dentro del archivo .env.

Despliegue con Docker compose

Paso 1:

Construir las imágenes Docker

```
$ docker compose -build
```

Este proceso puede tardar unos minutos en la primera ejecución.

Paso 2:

Levantar todos los servicios.

```
$ docker compose up -d
```

Paso 3:

Verificar que los contenedores estén en correcta ejecución.

```
$ docker compose ps
```

Esto debe mostrarse al ejecutar o anterior:

NAME	STATUS	PORTS
laesperanza-frontend	Up	0.0.0.0:80->80/tcp
laesperanza-backend	Up	0.0.0.0:3000->3000/tcp
laesperanza-db	Up	5432/tcp
laesperanza-nginx	Up	0.0.0.0:443->443/tcp

Paso 4:

Ejecutar las migraciones de base de datos

```
$ docker compose exec backend npx prisma migrate deploy
```

Paso 5:

```
$ docker compose exec backend npm run seed
```

Esto creara el catálogo de productos y un usuario administrador inicial.

Verificar la instalación

Verificar el frontend

```
Http://localhost
```

Al realizar lo anterior debe mostrarse la pantalla de inicio del sistema la esperanza.

Verificar el backend

Desde la terminal ejecute lo siguiente:

```
$ curl http://localhost:3000/api/v1/health
```

Respuesta esperada:

```
{“sucess”: true, “message”: “API La esperanza funcionando correctamente”,  
“versión”: “1.0.0” }
```

Verificar la base de datos

```
$ docker compose exec db psql -U laesperanza_user -d laesperanza_db -c “/dt”
```

Debe listar las 12 tablas del sistema

Comandos para la gestión

Componente	Versión Mínima	Notas
Detener servicios	docker compose stop	Detiene sin eliminar contenedores
Apagar y limpiar	docker compose down	Elimina contenedores (datos persisten)
Apagar y borrar datos	docker compose down -v	Elimina también los volúmenes
Ver logs en tiempo real	docker compose logs -f	Ctrl+C para salir
Logs de un servicio	docker compose logs -f backend	Reemplazar backend por el servicio deseado
Reiniciar un servicio	docker compose restart backend	Sin afectar los demás servicios
Actualizar el sistema	git pull && docker compose up -d --build	Descarga cambios y reconstruye

Solución de problemas comunes

Puerto 80 en uso

Error bind: adress already in use para el puerto 80

Solución: Identificar y detener el proceso que ocupa el puerto anterior:

```
$ sudo lsof -i :80  
$ sudo systemctl stop apache2 (si es Apache)  
$ sudo systemctl stop Nginx (Si es Nginx del host)
```

La base de datos no iniciales

Si el contenedor laesperanza-db aparece en estado "Exit", revisar logs:

```
$ docker compose logs db
```

Generalmente causado debido a que permisos en volumen o contraseña con caracteres especiales en el .env.

Erro durante las migraciones de prisma

```
$ docker compose exec backend npx prisma migrate reset  
$ docker compose exec backend npx prisma migrate deploy
```

El frontend muestra error con conexión de la API

Verificar que la variable "VITE_API_URL", en el .env apunta a la URL correcta del backend y reconstruir la imagen del frontend:

```
$ docker compose up -d --build frontend
```

Guía de lanzamiento

Resumen del proceso

Esta guía llevará desde cero hasta tener el sistema la esperanza corriendo en cualquier máquina, el proceso completo toma entre 15 a 25 minutos dependiendo de la conexión a internet.

#	Etapas	Qué se hace
1	Instalar requisitos	Docker Desktop y Git en tu computadora
2	Obtener el proyecto	Descargar y descomprimir el archivo laesperanza.zip
3	Configurar variables	Crear el archivo .env con contraseñas y configuración
4	Construir e iniciar	docker compose up — descarga imágenes y levanta todo
5	Inicializar la BD	Aplicar migraciones y cargar datos iniciales (seed)
6	Verificar y probar	Abrir el navegador y probar con las cuentas de prueba

Requisitos previos

Se necesitar instalar 2 herramientas antes de todo. Todo lo demás (Node.js, PostgreSQL, Nginx) corre dentro de Docker automáticamente.

Docker desktop

Docker es el motor que corre todos los servicios del sistema en contenedores aislados.

Sistema	Instrucciones
Windows 10/11	1. Ir a https://docker.com/products/docker-desktop 2. Descargar Docker Desktop para Windows 3. Ejecutar el instalador 4. Reiniciar la computadora 5. Abrir Docker Desktop y esperar a que el ícono de ballena quede verde
macOS	1. Ir a https://docker.com/products/docker-desktop 2. Descargar según tu chip: Intel o Apple Silicon (M1/M2/M3) 3. Arrastrar Docker.app a Aplicaciones 4. Abrirlo y aceptar permisos
Ubuntu/Linux	Ejecutar en terminal: <code>curl -fsSL https://get.docker.com sh && sudo usermod -aG docker \$USER && newgrp docker</code>

Git

Git es opcional, si descarga el .ZIP directamente, si no lo tiene, descárguelo desde: <http://git-scm.com/downloads> e instalarlo con las opciones por defecto

Obtener el proyecto

Descargar el archivo .zip

Descargue el archivo .zip → extraer todo

Descomprimir

Hacer clic derecho sobre el zip luego extraer todo

Abrir una terminal dentro de la carpeta

Navegar hacia dentro de la carpeta que dejó el archivo .zip

Configurar variables de entorno

El archivo .env contiene las configuraciones del sistema (contraseñas de BD, claves JWT, etc), deberá de ser creado antes de arrancar.

Crear el archivo .env

```
#macOS/Linux  
Cp .env.example .env  
  
#Windows  
Copy .env.example .env
```

Editar el archivo .env

Abrir el archivo .env con cualquier editor de texto y luego modificar obligatoriamente las siguientes líneas:

```
# cambiar las siguientes 3 lineas
POSTGRES_PASSWORD=TuPasswordSeguro202!

# Debe coincidir con POSTGRES_PASSWORD en la URL:
DATABASE_URL=postgresql://admin:TuPasswordSeguro202!@db:5432/laesperanza_db

# Clave secreta para JWT — mínimo 32 caracteres, cualquier texto largo:
JWT_SECRET=miClaveSecretaMuyLargaYSegura2026LaEsperanza

#las siguientes pueden quedar sin modificar
POSTGRES_DB=laesperanza_db
POSTGRES_USER=admin
JWT_EXPIRES_IN=7d
PORT=3000
NODE_ENV=production
FRONTEND_URL=http://localhost
```

Construir e iniciar el sistema

Este paso descarga todas las imágenes de Docker y construye los contenedores, la primera vez puede llegar a tardar varios minutos, esto dependerá de la conexión a internet.

Luego, se mostrarán los siguientes mensajes:

```
[+] up 5/6
✓ Image laesperanza-frontend      Built
✓ Image laesperanza-backend      Built
✓ Network laesperanza_default    Created
⚡ Container laesperanza-db       Waiting
✓ Container laesperanza-backend  Created
✓ Container laesperanza-frontend Created
```

Aplicar migraciones

Las migraciones crean la estructura de las 12 tablas de PostgreSQL:

```
Docker compose exec backend npx prisma migrate deploy
```

```
#deberá de ver lo siguiente
# Applying migration "20250501000000_init"
# All migrations have been successfully applied
```

Cargas los datos iniciales con la semilla (seed)

La seed crea el catálogo de productos y las 3 cuentas de prueba:

```
yovani > laesperanza > main > ?4 > 4.685s > docker
r compose exec backend npx prisma db seed
Running seed command `node prisma/seed.js` ...
🌱 Iniciando seed...
✅ Seed completado.

Cuentas de prueba (contraseña: Admin1234!):
🌱 Productor: productor@laesperanza.com
👤 Comprador: comprador@laesperanza.com
🏛️ Asociación: asociacion@laesperanza.com

🌱 The seed command has been executed.
```

Verificar que todo funciona correctamente

Abrir el sistema en el navegador

Abrir cualquier navegador de su preferencia y navegar hacia:

<http://localhost>

Deberá de visualizar la pantalla de login del sistema la esperanza con el formulario de inicio de sesión.

Iniciar sesión con las cuentas de prueba

Rol	Email	Contraseña	Acceso a
Productor	productor@laesperanza.com	Admin1234!	Publicaciones, negociaciones
Comprador	comprador@laesperanza.com	Admin1234!	Marketplace, compras
Asociación	asociacion@laesperanza.com	Admin1234!	Administración

Verificar que la API responda correctamente

En otra pestaña del navegador, verifique el health check de la API:

```
# En el navegador ir hacia:  
http://localhost:3000/api/v1/health  
  
#Deberá ver.  
{ "status": "ok", "timestamp": "2025-05-10T..." }
```

Comandos para la gestión diaria

Acción	Comando
Iniciar el sistema	docker compose up -d
Detener el sistema	docker compose down
Reiniciar un servicio	docker compose restart backend
Ver logs del backend	docker compose logs -f backend
Ver logs de la base de datos	docker compose logs -f db
Ver logs del frontend	docker compose logs -f frontend
Ver estado de servicios	docker compose ps
Eliminar todo y empezar de cero	docker compose down -v
Acceder al shell del backend	docker compose exec backend sh
Acceder a PostgreSQL	docker compose exec db psql -U admin -d laesperanza_db

Flujo de prueba completo

Siga los siguientes pasos para verificar que todos los módulos funcionan correctamente:

Como productor

1. Inicie sesión con productor@laesperanza.com / Admin1234!
2. En el dashboard, haga clic en “Nueva publicación”
3. Complete el formulario: seleccione “Maíz”, escriba un titulo, precio y cantidad.

4. Haga clic en “Publicar oferta” y verifique que aparece dentro de “Mis publicaciones”

Como comprador

1. Abra una ventana y navegue hacia <http://localhost>
2. Inicie sesión con: comprador@laesperanza.com / Admin1234!
3. Vaya a la publicación que creo anteriormente y haga clic sobre ella
4. Haga clic en “Iniciar negociaciones”, ingrese la cantidad y las condiciones que desee.
5. Verifique que aparece en “Negociaciones” con estado “pendiente”

Como productor (respuesta)

1. Vuelva a la ventana “productor” deberá de aparecer una nueva notificación.
2. Vaya a “negociaciones”, entre a la solicitud y acepte o negocie un precio para acordar el mismo.
3. Envíe un mensaje de prueba en el chat.
4. Cree una entrega desde el botón “gestionar entrega”.
5. Confirmar la entrega como productor.

Como comprador (completar pedido)

1. Vuelva a la ventana del comprador y navegue hacia entregas.
2. Confirme la entrega como comprador.
3. Verifique que la negociación para automáticamente a estado “completada”.
4. Registre un pago en el módulo de pagos.

Referencia rápida de puertos y servicios

Puerto	Servicio	Contenedor	Para qué sirve
80	Nginx	laesperanza-frontend	Acceso al sistema desde el navegador
3000	Node.js API	laesperanza-	API REST — endpoints del

Puerto	Servicio	Contenedor	Para qué sirve
		backend	sistema
5432	PostgreSQL	laesperanza-db	Base de datos (solo accesible internamente)

Historial de tareas

Este documento registra, el historial completo y detallado de las tareas realizadas durante el desarrollo de todo el sistema a lo largo de estas 3 frases, incluye la identificación de cada tarea, responsable, fechas, estado y entregable asociado.

Resumen general

Total de tareas: 37

Completadas: 37

Fase	Tareas	Completadas	Período
Fase I — Requisitos	7	7	Enero 2025
Fase II — Diseño	5	5	Febrero 2025
Fase III — Implementación	14	14	Marzo – Abril 2025
Fase III — Documentación	11	11	Mayo 2025
TOTAL	37	37	Ene – May 2025

Detalle de tareas por fase

Fase 1 requisitos

ID	Tarea	Responsable	Inicio	Fin	Estado	Entregables
T-001	Relevamiento de necesidades del sistema con los stakeholders	Equipo	07/01/2025	10/01/2025	Completada	Actas de reunión
T-002	Definición de los 25 requisitos funcionales (RF-01 al RF-25)	Equipo	11/01/2025	14/01/2025	Completada	Documento de RF
T-003	Definición de los 10 requisitos no funcionales	Equipo	14/01/2025	15/01/2025	Completada	Documento de RNF
T-004	Identificación de actores: productor, comprador,	Equipo	15/01/2025	15/01/2025	Completada	Documento de actores

ID	Tarea	Responsable	Inicio	Fin	Estado	Entregables
	asociación					
T-005	Elaboración de 25 casos de uso (CU-01 al CU-25)	Equipo	16/01/2025	20/01/2025	Completada	Catálogo de CU
T-006	Diseño del diagrama Entidad-Relación (8 entidades iniciales)	Equipo	21/01/2025	23/01/2025	Completada	Diagrama ER v1
T-007	Creación de mockups para 13 pantallas del sistema (v2)	Equipo	24/01/2025	31/01/2025	Completada	Mockups v2

Fase 2 diseño

ID	Tarea	Responsable	Inicio	Fin	Estado	Entregables
T-008	Elaboración del documento DERCAS (22 RF detallados)	Equipo	03/02/2025	07/02/2025	Completada	Documento DERCAS
T-009	Definición de 14 requisitos no funcionales detallados	Equipo	07/02/2025	08/02/2025	Completada	RNF detallados
T-010	Elaboración de 20 diagramas de actividad	Equipo	10/02/2025	17/02/2025	Completada	Diagramas de actividad
T-011	Diseño del diagrama de componentes	Equipo	18/02/2025	19/02/2025	Completada	Diagrama de componentes
T-012	Creación del prototipo interactivo HTML (13 pantallas)	Equipo	20/02/2025	28/02/2025	Completada	Prototipo interactivo HTML

Fase 3 implementación

ID	Tarea	Responsable	Inicio	Fin	Estado	Entregable
T-013	Diseño del esquema SQL final (12 tablas, triggers, vistas)	Equipo	03/03/2025	06/03/2025	Completada	database/schema.sql
T-014	Configuración de Docker Compose (frontend, backend, PostgreSQL, Nginx)	Equipo	07/03/2025	08/03/2025	Completada	docker-compose.yml
T-015	Configuración de Prisma ORM y esquema schema.prisma	Equipo	09/03/2025	10/03/2025	Completada	prisma/schema.prisma
T-016	Implementación del backend base (Express, estructura en capas)	Equipo	11/03/2025	13/03/2025	Completada	backend/src/
T-017	Implementación del módulo de autenticación JWT	Equipo	14/03/2025	17/03/2025	Completada	routes/auth, controllers/auth
T-018	Implementación de CRUD de publicaciones	Equipo	18/03/2025	21/03/2025	Completada	routes/publicaciones
T-019	Implementación del módulo de negociaciones y mensajería	Equipo	24/03/2025	28/03/2025	Completada	routes/negociaciones, mensajes
T-020	Implementación del módulo de entregas y confirmación doble	Equipo	31/03/2025	03/04/2025	Completada	routes/entregas
T-021	Implementación del módulo de pagos	Equipo	04/04/2025	06/04/2025	Completada	routes/pagos
T-022	Implementación del módulo de notificaciones (triggers automáticos)	Equipo	07/04/2025	09/04/2025	Completada	routes/notificaciones
T-023	Implementación del frontend base: Login y Registro	Equipo	10/04/2025	14/04/2025	Completada	Login.jsx, Registro.jsx

ID	Tarea	Responsable	Inicio	Fin	Estado	Entregable
T-024	Configuración de estado global con Zustand y cliente Axios	Equipo	14/04/2025	15/04/2025	Completada	store/auth.store.js, api/client.js
T-025	Implementación de las 10 páginas frontend restantes	Equipo	16/04/2025	30/04/2025	Completada	pages/ (10 componentes)
T-026	Integración y pruebas de sistema completo (end-to-end)	Equipo	28/04/2025	02/05/2025	Completada	Sistema integrado y funcional

Fase 3 documentación

ID	Tarea	Responsable	Inicio	Fin	Estado	Entregable
T-027	Elaboración del Diccionario de Datos (12 tablas)	Equipo	05/05/2025	05/05/2025	Completada	Diccionario_Datos.docx
T-028	Elaboración de la Documentación de API REST	Equipo	05/05/2025	06/05/2025	Completada	Documentacion_API.docx
T-029	Elaboración de la Guía de Instalación	Equipo	06/05/2025	06/05/2025	Completada	Guia_Instalacion.docx
T-030	Elaboración de la Guía de Entorno de Desarrollo	Equipo	06/05/2025	06/05/2025	Completada	Guia_Entorno_Desarrollo.docx
T-031	Elaboración del documento de Control de Versiones Gitflow	Equipo	07/05/2025	07/05/2025	Completada	Gitflow.docx
T-032	Elaboración del Plan de Pruebas	Equipo	07/05/2025	07/05/2025	Completada	Plan_Pruebas.docx
T-033	Elaboración de los 37 Casos de Prueba y ejecución	Equipo	08/05/2025	09/05/2025	Completada	Casos_Prueba.docx
T-034	Elaboración de Release Notes v1.0.0	Equipo	09/05/2025	09/05/2025	Completada	ReleaseNotes_v1.0.0.docx

ID	Tarea	Responsable	Inicio	Fin	Estado	Entregable
T-035	Elaboración de la Matriz de Trazabilidad (25 RF)	Equipo	10/05/2025	10/05/2025	Completada	Matriz_Trazabilidad.docx
T-036	Elaboración del Historial de Tareas	Equipo	10/05/2025	10/05/2025	Completada	Historial_Tareas.docx
T-037	Revisión final de documentos y empaquetado de entrega	Equipo	12/05/2025	12/05/2025	Completada	Entrega final Proyecto III

Infraestructura del sistema

Esta parte del documento describe la infraestructura tecnológica sobre la que opera el sistema, cubre la contenerización con Docker, la orquestación con Docker compose, la configuración de la red interna, los volúmenes de datos, las variables de entorno y las consideraciones de seguridad y respaldo.

La infraestructura fue planeada y diseñada bajo los principios de reproducibilidad, esto significa que cualquier entorno producirá el mismo resultado, aislamiento es decir, cada servicio corre en su propio contenedor para evitar conflictos, y la simplicidad operaria que se traduce en que un solo comando levanta todo el sistema.

Resumen de la infraestructura

Componente	Tecnología	Versión	Descripción
Motor de contenedores	Docker	24.0+	Crea y ejecuta los contenedores aislados
Orquestación	Docker Compose	2.20+	Coordina los 3 servicios con un solo comando
Servidor web	Nginx	Alpine	Proxy inverso y servidor de archivos estáticos
Runtime backend	Node.js	20 LTS	Ejecuta el servidor Express de la API
Base de datos	PostgreSQL	16	Motor relacional principal del sistema
Red interna	Docker Network	Bridge	Comunicación privada entre contenedores
Persistencia	Docker Volume	-	Almacenamiento persistente de la base de datos
Sistema operativo	Alpine Linux	3.18+	Imagen base mínima para todos los contenedores

Contenedor del sistema

El sistema está compuesto por exactamente 3 contenedores Docker, cada uno con una responsabilidad única pero sobretodo bien definida, los cuales son los siguientes:

1. laesperanza-frontend

Imagen base: nginx:alpine

Puertos: 80:80 (HTTP)

Rol: Sirve el build estático de React, actúa como proxy inverso redirigiendo /api/* al backend.

2.laesperanza-backend

Imagen base: node:20-alpine

Puertos: 3000:3000 (de la API rest)

Rol: Ejecuta el servidor express con todos los endpoints de la API, además conecta a PostgreSQL vía prisma ORM.

3.laesperanza.db

Imagen base: postgres:16-alpine

Puertos: 5432 (solo parte interna)

Rol: Motor de nuestra base de datos en PostgreSQL, solo es accesible desde la red interna de Docker, por lo mismo no es expuesto al exterior.

Orquestación con Docker compose

El archivo docker-compose.yml, en la raíz del proyecto define y coordina los 3 servicios, así:

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./database/schema.sql:/docker-entrypoint-initdb.d/
    healthcheck:
      test: pg_isready -U admin
      interval: 10s timeout: 5s retries: 5

  backend:
    build: ./backend
    depends_on:
      db:
        condition: service_healthy
    environment:
      DATABASE_URL, JWT_SECRET, PORT, NODE_ENV

  frontend:
    build: ./frontend
    depends_on:
      - backend
    ports:
      - '80:80'

volumes:
  pgdata:
```

Dockerfiles por servicio

Frontend

El frontend usa un build de 2 etapas para producir una imagen final mínima, así:

```
#Etapa 1: construcción del build de React
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

#Etapa 2: imagen final solo con nginx y los files static
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY ../nginx/nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Ventaja del build

La imagen final no contiene Node.js ni el código fuente.
Tamaño reducido: 25 MB vs 900 MB con Node.js incluido.
Superficie de ataque mínima en producción.

Backend

```
#dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . . .
RUN npx prisma generate
EXPOSE 3000
#al iniciar aplicar las migraciones y arranque el servidor
CMD ["sh", "-c", "npx prisma migrate deploy && node src/index.js"]
```

Red interna del Docker

Docker compose crea automáticamente una red bridge privada llamada "laesperanza_default", que conecta los contenedores, dentro de esta red los contenedores se comunican usando el nombre del servicio como hostname, así:

Origen	Destino	Hostname usado	Puerto	Propósito
frontend (Nginx)	backend (Node.js)	backend	3000	Proxy de /api/* al servidor Express
backend (Node.js)	db (PostgreSQL)	db	5432	Consultas SQL vía Prisma ORM
Exterior (navegador)	frontend (Nginx)	localhost	80	Acceso del usuario al sistema

Volúmenes y persistencia de datos

Docker usa un volumen nombrado para garantizar que los datos de PostgreSQL sobrevivan de reinicios del contenedor.

Volumen	Montado en	Tipo	Propósito
pgdata	/var/lib/postgresql/data	Volumen Docker	Datos de PostgreSQL. Persiste entre reinicios.
./database/ (bind)	docker-entrypoint-initdb.d/	Bind mount	Schema SQL ejecutado automáticamente al crear la BD.
./backend/ (dev)	app/	Bind mount	Solo en desarrollo: permite hot-reload sin reconstruir.

Backup manua de la base de datos

#creación del backup

```
Docker compose exec db pg_dump -U admin laesperanza_db > backup_$(date +%Y%m%d).sql
```

#restauración del backup

```
Docker compose exec -T db psql -U admin laesperanza_db < backup_20260510.sql
```

Variables de entorno

Toda la configuración sensible se centraliza en el archivo .env en la raíz del proyectom este archivo nunca debe subirse al repositorio de GIT.

Variable	Valor de ejemplo	Servicio	Descripción
POSTGRES_DB	laesperanza_db	DB	Nombre de la base de datos
POSTGRES_USER	admin	DB	Usuario administrador de PostgreSQL
POSTGRES_PASSWORD	S3gura!2025	DB	Contraseña del usuario (mín. 12 chars)
DATABASE_URL	postgresql://...	Backend	URL completa de conexión para Prisma
JWT_SECRET	clave_32+_chars	Backend	Clave para firmar tokens JWT (mín. 32 chars)
JWT_EXPIRES_IN	7d	Backend	Tiempo de expiración del token JWT
PORT	3000	Backend	Puerto en que escucha Express
NODE_ENV	production	Backend	Entorno de ejecución
FRONTEND_URL	http://localhost	Backend	URL del frontend permitida por CORS

Configuración de Nginx

Nginx cumple dos roles simultáneos: servidor de archivos estático para el frontend Reacty como proxy inverso para redigir las solicitudes de API al backend.

```

Server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;

  gzip on;
  gzip_types text/css application/javascript application/json;

  location / {
    try_files $uri $uri/ /index.html;
  }

  location /api/ {
    proxy_pass http://backend:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
  }

  location ~* \.(js|css|png|jpg|svg|woff2)$ {
    expires 1y;
    add_header Cache-Control 'public, immutable';
  }
}

```

Mapa de los puertos

Puerto	Contenedor	Protocolo	Exposición	Descripción
80	frontend	HTTP	Público	Acceso del usuario al sistema desde el navegador
443	frontend	HTTPS	Público (prod)	<i>SSL/TLS — configurar con Certbot en producción real</i>
3000	backend	HTTP	Interno	API REST — solo accesible dentro de la red Docker
5432	db	TCP	Interno	PostgreSQL — solo accesible por el backend, no expuesto

Secuencia de arranque

Docker compose garantiza el orden correcto de inicio mediante depends_on y health checks, así:

Pas o	Servicio	Qué ocurre
1	laesperanza-db	PostgreSQL inicia. Health check verifica cada 10s con pg_isready hasta responder 'healthy'.
2	laesperanza-backend	Espera a que db esté 'healthy'. Ejecuta 'prisma migrate deploy' para aplicar migraciones. Inicia Express en puerto 3000.
3	laesperanza-frontend	Espera a que backend esté activo. Nginx sirve el build de React en puerto 80 y configura el proxy a backend:3000.

Requisitos mínimos del servidor

Recurso	Mínimo	Recomendado	Notas
CPU	1 vCPU	2 vCPU	Node.js es single-thread pero PostgreSQL se beneficia de más cores
RAM	2 GB	4 GB	PostgreSQL requiere ~512MB, Node.js ~256MB, Nginx ~50MB

Recurso	Mínimo	Recomendado	Notas
Disco	10 GB	20 GB	Imágenes Docker ~3GB + datos de BD + logs
Sistema operativo	Linux	Ubuntu 22.04 LTS	macOS y Windows con Docker Desktop también funcionan
Docker Engine	24.0	24.0+	Versiones anteriores pueden tener incompatibilidades con Compose v2
Docker Compose	2.20	2.20+	Sintaxis de Compose v2 requerida (sin guión en comandos)
Ancho de banda	10 Mbps	50 Mbps	Para descarga de imágenes Docker la primera vez

Consideraciones de seguridad

Medidas implementadas

Contraseñas hasheadas con bcrypt (factor de coste 10) — nunca se almacenan en texto plano

Tokens JWT firmados con clave secreta configurable y expiración de 7 días

CORS restrictivo: solo acepta solicitudes del dominio configurado en FRONTEND_URL

PostgreSQL no expuesto al exterior — solo accesible dentro de la red Docker

Variables sensibles en .env — excluido del repositorio con .gitignore

Control de acceso basado en roles (RBAC) en todos los endpoints protegidos

Imágenes Alpine Linux: superficie de ataque mínima por tamaño reducido

Recomendaciones para producción real

Configurar HTTPS con Certbot/Let's Encrypt en el servidor Nginx

Usar un gestor de secretos (AWS Secrets Manager, HashiCorp Vault) en lugar de .env

Habilitar logs de auditoría en PostgreSQL (log_statement = 'all')

Configurar backups automáticos diarios de la base de datos

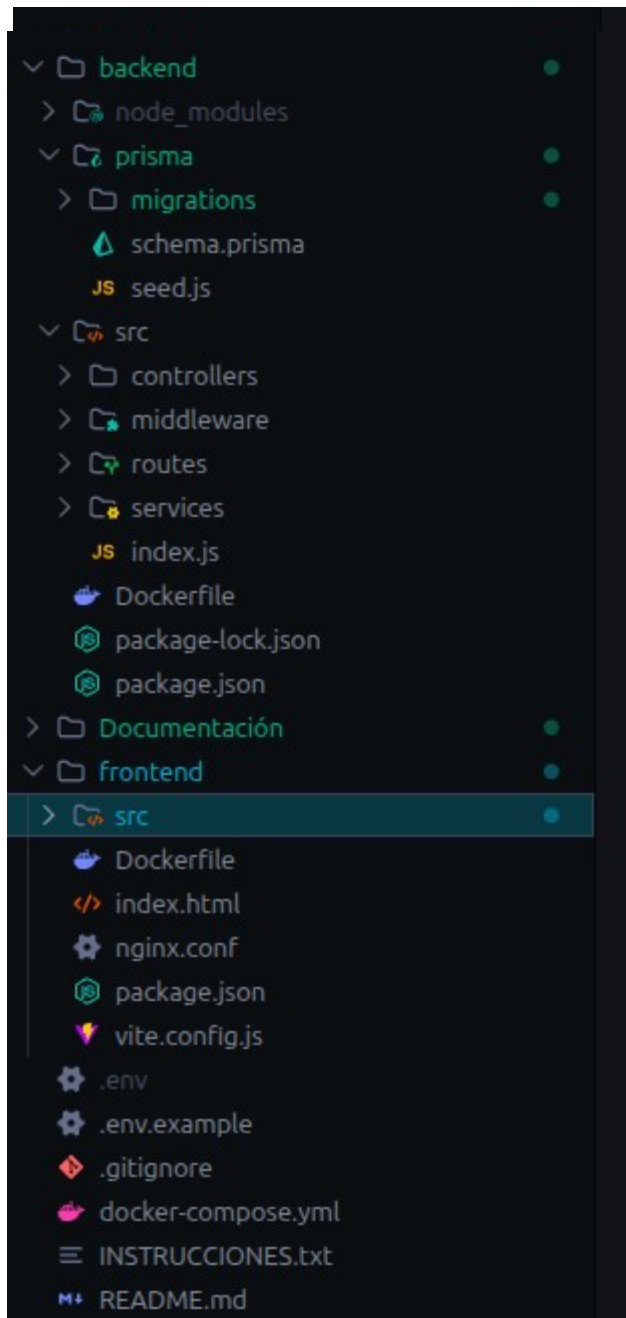
Restringir el acceso SSH al servidor con claves públicas únicamente

Implementar rate limiting en Nginx para prevenir ataques de fuerza bruta

Comandos para la operación y mantenimiento del sistema

Operación	Comando
Iniciar todos los servicios	<code>docker compose up -d</code>
Iniciar y reconstruir imágenes	<code>docker compose up --build -d</code>
Detener sin eliminar datos	<code>docker compose down</code>
Detener y eliminar todos los datos	<code>docker compose down -v</code>
Ver estado de contenedores	<code>docker compose ps</code>
Ver logs en tiempo real	<code>docker compose logs -f</code>
Ver logs solo del backend	<code>docker compose logs -f backend</code>
Reiniciar un servicio	<code>docker compose restart backend</code>
Aplicar migraciones de BD	<code>docker compose exec backend npx prisma migrate deploy</code>
Cargar datos iniciales (seed)	<code>docker compose exec backend npx prisma db seed</code>
Abrir consola de PostgreSQL	<code>docker compose exec db psql -U admin -d laesperanza_db</code>
Crear backup de la BD	<code>docker compose exec db pg_dump -U admin laesperanza_db > backup.sql</code>
Restaurar backup	<code>docker compose exec -T db psql -U admin laesperanza_db < backup.sql</code>
Ver uso de recursos	<code>docker stats</code>
Eliminar imágenes no usadas	<code>docker image prune -f</code>

Estructura de archivos de infraestructura



Matriz de trazabilidad

Función

la matriz de trazabilidad, tiene como función relacionar cada requisito funcional (RF), con su caso de uso (CU) respectivo, módulo del sistema, endpoints de la API REST implementados, tablas de la base de datos involucradas y casos de prueba que validaron su correcto funcionamiento, todos estos datos fueron obtenidos en las 2 entregas anteriores, en donde se recabó las requisitos con los que debía de contar el sistema.

Este documento permite verificar que cada requisito está completamente implementado, probado y documentado, cerrando el ciclo de calidad del Proyecto III.

Resumen de la cobertura

Requisitos funcionales totales: 25

Implementados: 25

Con casos de prueba: 18

Cobertura: 100%

Módulo	RF Total	Implemen tados	Cobertura
Autenticación	3	3	100%
Productores	2	2	100%
Compradores	1	1	100%
Publicaciones	3	3	100%
Negociaciones	3	3	100%
Mensajería	2	2	100%
Entregas	2	2	100%
Pagos	2	2	100%
Asociaciones	2	2	100%
Catálogo	1	1	100%
Notificaciones	2	2	100%
Seguridad	2	2	100%
TOTAL	25	25	100%

Leyenda clasificando los módulos por colores

Autenticación
Productores
Compradores
Publicaciones
Negociaciones
Mensajería
Entregas
Pagos
Asociaciones
Catálogo
Notificaciones
Seguridad

Conversiones de la matriz

RF: Requisitos funcionales

CU: Casos de uso

CP: Casos de prueba

✓: Implementado y verificado

--: Sin CP directo asignado (validado por integración)

Matriz de trazabilidad completa

RF	Descripción	Prioridad	CU	Módulo	Endpoint(s)	Tabla(s) BD	CP
RF-01	Registro de nuevo usuario	Alta	CU-01	Autenticación	POST /auth/register	usuarios, productores / compradores / asociaciones	CP-01, CP-02
RF-02	Inicio de sesión con JWT	Alta	CU-02	Autenticación	POST /auth/login GET /auth/me	usuarios	CP-03, CP-04, CP-05, CP-06
RF-03	Cierre de sesión	Media	CU-03	Autenticación	POST /auth/logout	(manejo en cliente)	—
RF-04	Gestión de perfil de productor	Alta	CU-04	Productores	GET /productores/:id PUT /productores/:id	productores, usuarios	—
RF-05	Listar productores	Media	CU-05	Productores	GET /productores	productores	—
RF-06	Gestión de perfil de comprador	Alta	CU-06	Compradores	GET /compradores/:id PUT /compradores/:id	compradores, usuarios	—
RF-07	Crear publicación de oferta de venta	Alta	CU-07	Publicaciones	POST /publicaciones	publicaciones, productos, productores	CP-07, CP-08, CP-09
RF-08	Buscar y filtrar publicaciones	Alta	CU-08	Publicaciones	GET /publicaciones GET /publicaciones/:id	publicaciones, productos	CP-10, CP-13
RF-09	Gestionar estado de publicación	Media	CU-09	Publicaciones	PUT /publicaciones/:id PATCH /publicaciones/:id/estado DELETE /publicaciones/:id	publicaciones	CP-11, CP-12
RF-10	Iniciar negociación privada	Alta	CU-10	Negociaciones	POST /negociaciones	negociaciones, publicaciones,	CP-14, CP-15, CP-20

RF	Descripción	Prioridad	CU	Módulo	Endpoint(s)	Tabla(s) BD	CP
						compradores	
RF-11	Gestionar estado de negociación	Alta	CU-11	Negociaciones	PATCH /negociaciones/:id/estado	negociaciones	CP-16, CP-17, CP-18
RF-12	Consultar negociaciones propias	Alta	CU-12	Negociaciones	GET /negociaciones GET /negociaciones/:id	negociaciones	CP-19
RF-13	Enviar mensajes en negociación	Alta	CU-13	Mensajería	POST /negociaciones/:id/mensajes	mensajes, negociaciones	CP-21, CP-22, CP-24
RF-14	Leer mensajes de negociación	Media	CU-14	Mensajería	GET /negociaciones/:id/mensajes PATCH /negociaciones/:id/mensajes/leer	mensajes	CP-23
RF-15	Registrar entrega de producto	Alta	CU-15	Entregas	POST /entregas PUT /entregas/:id	entregas, negociaciones	CP-25, CP-26
RF-16	Confirmación doble de entrega	Alta	CU-16	Entregas	POST /entregas/:id/confirmar GET /entregas/:id	confirmaciones_entrega, entregas, negociaciones	CP-27, CP-28, CP-29, CP-30
RF-17	Registrar pago de negociación	Alta	CU-17	Pagos	POST /pagos GET /pagos GET /pagos/:id	pagos, negociaciones	CP-31, CP-33, CP-34
RF-18	Actualizar estado de pago	Media	CU-18	Pagos	PUT /pagos/:id	pagos	CP-32
RF-19	Registrar asociación de agricultores	Alta	CU-19	Asociaciones	POST /asociaciones	asociaciones, usuarios	—
RF-20	Gestionar miembros de asociación	Media	CU-20	Asociaciones	GET /asociaciones/:id PUT /asociaciones/:id	asociaciones, productores	—
RF-21	Gestionar catálogo de productos	Media	CU-21	Catálogo	GET /productos POST /productos PUT /productos/:id	productos	—
RF-22	Generación automática de notificaciones	Media	CU-22	Notificaciones	(trigger automático del backend)	notificaciones	CP-35
RF-23	Consultar y gestionar notificaciones	Baja	CU-23	Notificaciones	GET /notificaciones PATCH	notificaciones	CP-36, CP-37

RF	Descripción	Prioridad	CU	Módulo	Endpoint(s)	Tabla(s) BD	CP
					/notificaciones/:id/leer PATCH /notificaciones/leer-todas		
RF-24	Control de acceso basado en roles	Alta	CU-24	Seguridad	(middleware en todos los endpoints)	usuarios	CP-09, CP-15, CP-19, CP-22
RF-25	Privacidad de negociaciones	Alta	CU-25	Seguridad	(validación en negociaciones, mensajes, entregas)	negociaciones, mensajes, entregas	CP-19, CP-22, CP-30

Plan de pruebas

El plan de pruebas define la estrategia, alcance, recursos y cronograma de actividades de verificación y validación para el sistema, su objetivo es garantizar que dicho sistema cumpla con los requisitos funcionales y no funcionales ya establecidos en las fases previas.

Este documento aplica la versión 1.0.0 del sistema e incluye los niveles de prueba a ejecutar, los módulos bajo prueba, los criterios de entrada y salida y las métricas de calidad esperadas.

Identificación del proyecto

Campo	Detalle
Nombre del sistema	Sistema La Esperanza — Plataforma de Gestión Agrícola
Versión bajo prueba	1.0.0
Tipo de sistema	Aplicación web (SPA + API REST + PostgreSQL)
Equipo de pruebas	Equipo de desarrollo del Proyecto III
Fecha de inicio	Mayo 2025
Fecha de cierre	Mayo 2025
Documento base	Requisitos funcionales (25 RF) y no funcionales (10 RNF)

Alcance de las pruebas

Módulos bajo pruebas

Los siguientes módulos del sistema serán sometidos bajo pruebas:

#	Módulo	Descripción
1	Autenticación	Registro, login y gestión de sesión con JWT
2	Gestión de Usuarios	Perfiles de productores, compradores y asociaciones
3	Catálogo de Productos	CRUD del catálogo de productos agrícolas
4	Publicaciones	Creación, edición, estado y búsqueda de ofertas de venta
5	Negociaciones	Flujo completo de negociación entre productor y comprador

#	Módulo	Descripción
6	Mensajería	Envío y lectura de mensajes dentro de una negociación
7	Entregas	Registro y confirmación doble de entregas
8	Pagos	Registro y actualización de pagos por negociación
9	Notificaciones	Generación, lectura y gestión de alertas del sistema
10	Seguridad y Roles	Control de acceso por rol: productor, comprador, asociación

Niveles de pruebas

Se aplicarán los siguientes niveles de prueba de menor a mayor rubricidad:

Tipo de Prueba	Responsable	Herramienta	Objetivo	Cobertura Meta
Pruebas Unitarias	Equipo	Jest	Verificar funciones y métodos aislados del backend	≥ 70%
Pruebas de Integración	Equipo	Jest Supertest +	Verificar comunicación entre módulos y con la BD	≥ 60%
Pruebas de API	Equipo	Postman Thunder /	Validar todos los endpoints REST documentados	100% endpoints
Pruebas de Sistema	Equipo	Manual navegador +	Validar flujos completos del sistema end-to-end	Casos críticos
Pruebas de Aceptación	Product Owner	Manual	Confirmar que el sistema cumple los RF definidos	25 RF

Criterios para la entrada y salida

Criterios de entrada (inicio de pruebas)

El entorno de pruebas está configurado y disponible

El código fuente compila sin error alguno.

Las migraciones de la base de datos han sido aplicadas correctamente

Los casos de prueba han sido documentados y revisados

Los datos de la semilla de prueba están cargados dentro de la base de datos

Criterios para la salida (finalización de las pruebas)

100% de los casos de las pruebas críticos ejecutados correctamente
0 defectos de severidad critica sin resolver
Cobertura del código $\geq 70\%$ en pruebas unitarias del backend
Todos los 25 requisitos funcionales son validos
Reporte de resultados firmado por el responsable

Tipos de defectos y severidad

Severidad	Impacto	Descripción y ejemplos
Crítica	Sistema caído	El sistema no arranca, pérdida de datos, fallo de seguridad grave (ej. acceso sin autenticación).
Alta	Función principal	Un flujo principal no funciona (ej. no se puede crear una publicación, el login falla).
Media	Función secundaria	Funcionalidad secundaria con error o comportamiento incorrecto pero con workaround.
Baja	Cosmético/UX	Errores visuales, textos incorrectos, problemas de UX sin impacto funcional.

Entorno para las pruebas

Componente	Especificación
Sistema Operativo	Ubuntu 22.04 LTS / Windows 11 con WSL2
Servidor de pruebas	Docker Compose local (mismo que desarrollo)
Base de datos	PostgreSQL 16 — instancia laesperanza_test
Navegadores a probar	Google Chrome 120+, Mozilla Firefox 121+, Microsoft Edge 120+
Herramienta API	Postman v10 / Thunder Client
Framework de pruebas	Jest 29 + Supertest 6
Cobertura de código	jest --coverage (Istanbul)
Datos de prueba	Seed automatizado con npx prisma db seed

Estrategia de pruebas por módulo

Módulo	Unitaria	Integración	API	Sistema/Aceptación
Autenticación	✓	✓	✓	✓
Gestión de Usuarios	✓	✓	✓	✓
Catálogo de Productos	✓	—	✓	—
Publicaciones	✓	✓	✓	✓
Negociaciones	✓	✓	✓	✓
Mensajería	—	✓	✓	✓
Entregas	✓	✓	✓	✓
Pagos	✓	—	✓	✓
Notificaciones	—	✓	✓	—
Seguridad y Roles	✓	✓	✓	✓

Métricas para la valoración de la calidad

Métrica	Meta	Herramienta de medición
Cobertura de pruebas unitarias (backend)	≥ 70%	Jest --coverage
Casos de prueba ejecutados	100%	Registro manual en doc
Casos de prueba aprobados	≥ 95%	Registro manual en doc
Defectos críticos al cierre	0	Registro en tabla de defectos
Defectos altos sin resolver al cierre	0	Registro en tabla de defectos
Requisitos funcionales validados	25 / 25 (100%)	Matriz de trazabilidad
Tiempo de respuesta API (endpoints clave)	< 500 ms	Postman tests

Cronograma para las pruebas

#	Actividad	Responsable	Duración	Entregable
1	Preparación del entorno de pruebas	Desarrollador	1 día	Entorno configurado
2	Carga de datos de prueba (seed)	Desarrollador	0.5 días	BD de pruebas lista
3	Ejecución de pruebas unitarias	Desarrollador	2 días	Reporte Jest + cobertura
4	Ejecución de pruebas de integración	Desarrollador	1.5 días	Reporte de integración
5	Ejecución de pruebas de API (Postman)	Desarrollador	1.5 días	Colección Postman ejecutada
6	Pruebas de sistema (flujos end-to-end)	Equipo	2 días	Casos de prueba documentados
7	Pruebas de aceptación (validación RF)	Product Owner	1 día	Aprobación de requisitos
8	Registro y corrección de defectos	Desarrollador	2 días	Defectos resueltos
9	Reporte final de pruebas	Equipo	0.5 días	Documento de resultados

Distribución de roles y responsabilidades

Rol	Responsabilidades
Desarrollador / Tester	Diseñar y ejecutar pruebas unitarias, de integración y de API. Registrar y corregir defectos.
Líder de proyecto	Supervisar el cumplimiento del plan. Aprobar criterios de salida.
Product Owner / Cliente	Validar pruebas de aceptación. Confirmar que el sistema cumple los RF.

Riesgos para el plan de pruebas

Riesgo	Probabilidad	Impacto	Mitigación
Tiempo insuficiente para cubrir todos los casos	Media	Alta	Priorizar casos de flujos críticos. Automatizar lo más posible.
Entorno de pruebas inestable	Baja	Alta	Usar Docker Compose para garantizar reproducibilidad.
Defectos descubiertos tardíamente	Media	Media	Ejecutar pruebas desde las primeras iteraciones (CI).
Cambios de requisitos durante pruebas	Baja	Media	Congelar requisitos antes del ciclo de pruebas.

Release notes V1.0.0

información de la versión

Campo	Detalle
Versión	1.0.0
Tipo de release	Release inicial (GA — General Availability)
Fecha de liberación	Mayo 2025
Rama de origen	release/1.0.0 → main
Tag Git	v1.0.0
Autor	Equipo de desarrollo — Proyecto III
Estado	Estable — Listo para producción

Resumen

La versión v1.0.0 constituye el release inicial del sistema, una plataforma web que digitaliza y facilita la comercialización agrícola de un sector en específico, con soporte para asociaciones de agricultores como entidades organizadoras.

Esta primera versión libera el sistema completo con todas las funcionalidades core implementadas, probadas y documentadas respectivamente, lista para su despliegue en el entorno de producción mediante docker.

Nuevas funcionalidades

Como se trata de la versión primaria, no existen funcionalidades nuevas, esto porque prácticamente todo lo del sistema es nuevo.

Módulo de autenticación y usuarios

Registro de usuarios con roles diferenciados: productor, comprador y asociación.

Autenticación segura con JWT (JSON Web Tokens), expiración configurable.

Control de acceso basado en roles (RBAC) en todos los endpoints.

Perfiles extendidos por rol: datos comerciales para compradores, datos agrícolas para productores.

Módulo para las publicaciones

Creación de ofertas de venta por parte de productores con imagen, precio, cantidad y ubicación.

Búsqueda y filtrado de publicaciones por departamento, municipio, producto y rango de precio.

Gestión de estados de publicación: activa, pausada, cerrada, vencida.
Paginación en listados para manejo eficiente de grandes volúmenes de datos.

Módulo para las negociaciones en privado

Flujo de negociación privada entre comprador y productor sobre una publicación.
Gestión de estados: pendiente → en_proceso → aceptada → completada / rechazada / cancelada.
Registro de precio acordado, condiciones y fecha de entrega pactada.
Restricción de privacidad: solo los participantes pueden ver y actuar en una negociación.

Módulo para mensajería

Canal de mensajería privado dentro de cada negociación.
Marcado de mensajes como leídos individual y masivo.
Generación automática de notificación al recibir un nuevo mensaje.

Módulo para pagos

Registro de pagos por negociación con método, referencia y fecha.
Soporte para múltiples métodos: efectivo, transferencia, cheque, otro.
Gestión de estados de pago: pendiente, completado, fallido, reembolsado.

Módulo de notificaciones

Generación automática de alertas internas por eventos del sistema.
Tipos de notificación: nueva_negociacion, mensaje, confirmacion, sistema.
Marcado individual y masivo de notificaciones como leídas.

Infraestructura y DevOPS

Arquitectura de 5 capas: Frontend → Nginx → Backend → Prisma ORM → PostgreSQL 16.
Contenerización completa con Docker y orquestación con Docker Compose.
Variables de entorno centralizadas con .env.example documentado.
Esquema de base de datos con 12 tablas, triggers y vistas en PostgreSQL 16.
Migraciones versionadas con Prisma Migrate.

Mejoras técnicas

Implementación de Prisma ORM para abstracción y seguridad en acceso a datos.

Arquitectura backend en 4 capas (routes → controllers → services → repositories) para separación de responsabilidades.

Estado global en frontend con Zustand (librería ligera vs Redux).

Cliente HTTP centralizado con Axios e interceptores para manejo automático de tokens.

Configuración de CORS restrictiva para limitar acceso solo al dominio del frontend.

Correcciones de bugs

Al ser la primaria versión, los bugs no aplican correcciones de versiones anteriores porque no existen.

Requisitos del sistema

Componente	Requisito mínimo
Docker Engine	24.0 o superior
Docker Compose	2.20 o superior
RAM del servidor	2 GB mínimo (4 GB recomendado)
Disco	10 GB disponibles para imágenes y datos
Sistema Operativo	Linux (Ubuntu 22.04+), macOS 13+, Windows 11 con WSL2
Navegador cliente	Chrome 120+, Firefox 121+, Edge 120+
Puertos requeridos	80 (HTTP), 3000 (API), 5432 (BD — solo interno)

Vista de la base de datos para el sistema

-- SISTEMA LA ESPERANZA — Vistas de Base de Datos
-- Versión: 1.0.0 Mayo 2026
-- Motor: PostgreSQL 16
--
-- DOCUMENTACIÓN TÉCNICA
-- Estas vistas están definidas en el modelo de datos original
-- (Diagrama ER y schema.sql) pero NO fueron implementadas
-- en el schema de Prisma del proyecto.
-- Se documentan aquí para referencia en implementación futura.

```
CREATE OR REPLACE VIEW v_publicaciones_activas AS
SELECT
  p.id,
  p.titulo,
  p.precio_unitario,
  p.cantidad_disponible,
  p.unidad_medida,
  p.departamento,
  p.municipio,
  p.fecha_cosecha,
  p.imagen_url,
  p.created_at,
  pr.nombre      AS nombre_producto,
  pr.categoria   AS categoria_producto,
  u.nombre       AS nombre_productor,
  prod.calificacion AS calificacion_productor
FROM publicaciones p
JOIN productos   pr  ON p.producto_id = pr.id
JOIN productores prod ON p.productor_id = prod.id
JOIN usuarios    u   ON prod.usuario_id = u.id
WHERE p.estado = 'activa';

COMMENT ON VIEW v_publicaciones_activas IS
'Publicaciones activas con información combinada de producto y productor';

COMMENT ON COLUMN v_publicaciones_activas.nombre_productor IS
'Nombre del productor desde la tabla usuarios';
```

```
CREATE OR REPLACE VIEW v_negociaciones_resumen AS
SELECT
  n.id,
  n.estado,
  n.cantidad_solicitada,
  n.precio_acordado,
  n.fecha_entrega_acordada,
  n.created_at,
```

```

    p.titulo      AS titulo_publicacion,
    uc.nombre     AS nombre_comprador,
    up.nombre     AS nombre_productor
FROM negociaciones n
JOIN publicaciones p  ON n.publicacion_id = p.id
JOIN compradores c   ON n.comprador_id  = c.id
JOIN usuarios uc     ON c.usuario_id    = uc.id
JOIN productores pr  ON n.productor_id  = pr.id
JOIN usuarios up     ON pr.usuario_id   = up.id;

COMMENT ON VIEW v_negociaciones_resumen IS
'Resumen de negociaciones con participantes y publicación';

```

```

CREATE OR REPLACE VIEW v_entregas_pendientes AS
SELECT
    e.id,
    e.fecha_programada,
    e.fecha_realizada,
    e.lugar_entrega,
    e.estado      AS estado_entrega,
    e.notas,
    n.id          AS negociacion_id,
    n.estado      AS estado_negociacion,
    p.titulo      AS titulo_publicacion,
    up.nombre     AS nombre_productor,
    uc.nombre     AS nombre_comprador,
    (SELECT COUNT(*) FROM confirmaciones_entrega ce
     WHERE ce.entrega_id = e.id AND ce.confirmado = TRUE) AS confirmaciones_recibidas
FROM entregas e
JOIN negociaciones n  ON e.negociacion_id = n.id
JOIN publicaciones p  ON n.publicacion_id = p.id
JOIN productores pr   ON n.productor_id  = pr.id
JOIN usuarios up      ON pr.usuario_id   = up.id
JOIN compradores c    ON n.comprador_id  = c.id
JOIN usuarios uc      ON c.usuario_id    = uc.id;

COMMENT ON VIEW v_entregas_pendientes IS
'Entregas con detalle de negociación, participantes y conteo de confirmaciones';

```

```

CREATE OR REPLACE VIEW v_pagos_resumen AS
SELECT
    p.estado,
    COUNT(*)           AS cantidad_pagos,
    SUM(p.monto)       AS monto_total,
    MIN(p.fecha_pago)  AS primer_pago,
    MAX(p.fecha_pago)  AS ultimo_pago,

```

```
AVG(p.monto)          AS monto_promedio
FROM pagos p
GROUP BY p.estado
ORDER BY p.estado;

COMMENT ON VIEW v_pagos_resumen IS
'Resumen de pagos agrupados por estado con métricas agregadas';
```

-- NOTAS DE IMPLEMENTACIÓN

--Para agregar estas vistas al proyecto real se debe:

-- 1. Crear un archivo de migración SQL raw en Prisma:

-- prisma/migrations/<timestamp>_vistas/migration.sql

-- 2. O ejecutar manualmente contra la BD:

-- psql -U <user> -d <database> -f Vistas_DB_LaEsperanza.sql

-- 3. Si se usa Prisma, agregar en schema.prisma:

```
-- model v_publicaciones_activas {
--   // Definición como view (Prisma 5+ soporta views)
--   @@map("v_publicaciones_activas")
-- }
```

-- 4. Extensión recomendada (documentada pero no implementada):

-- CREATE EXTENSION IF NOT EXISTS "pg_trgm";

Script para la creación de la base de datos

-- SISTEMA LA ESPERANZA script de creación de Base de Datos

-- Motor: PostgreSQL 16

CREATE EXTENSION IF NOT EXISTS "pg_trgm";

```
CREATE TYPE rol_usuario AS ENUM ('productor', 'comprador', 'asociacion');
CREATE TYPE estado_publicacion AS ENUM ('activa', 'pausada', 'cerrada', 'vencida');
CREATE TYPE estado_negociacion AS ENUM ('pendiente', 'en_proceso', 'aceptada',
'rechazada', 'completada', 'cancelada');
CREATE TYPE estado_entrega AS ENUM ('pendiente', 'en_transito', 'entregado',
'con_problema');
CREATE TYPE estado_pago AS ENUM ('pendiente', 'completado', 'fallido',
'reembolsado');
```

-- TABLA: usuarios

```
CREATE TABLE usuarios (
  id SERIAL PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  rol rol_usuario NOT NULL,
  telefono VARCHAR(20),
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()
);
```

CREATE INDEX idx_usuarios_email ON usuarios(email);

CREATE INDEX idx_usuarios_rol ON usuarios(rol);

-- TABLA: asociaciones

```
CREATE TABLE asociaciones (
  id SERIAL PRIMARY KEY,
  usuario_id INTEGER NOT NULL UNIQUE REFERENCES usuarios(id) ON DELETE
CASCADE,
  nombre VARCHAR(200) NOT NULL,
  nit VARCHAR(20) UNIQUE,
  municipio VARCHAR(100),
  departamento VARCHAR(100),
  descripcion TEXT,
```

```
    activa    BOOLEAN    NOT NULL DEFAULT TRUE,  
    created_at  TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

-- TABLA: productores

```
CREATE TABLE productores (  
    id          SERIAL      PRIMARY KEY,  
    usuario_id  INTEGER     NOT NULL UNIQUE REFERENCES usuarios(id) ON DELETE  
CASCADE,  
    asociacion_id INTEGER    REFERENCES asociaciones(id),  
    dpi         VARCHAR(20)  UNIQUE,  
    municipio   VARCHAR(100),  
    departamento VARCHAR(100),  
    hectareas   DECIMAL(8,2),  
    descripcion TEXT,  
    calificacion DECIMAL(3,2) NOT NULL DEFAULT 0.00,  
    created_at  TIMESTAMP    NOT NULL DEFAULT NOW()  
);
```

```
CREATE INDEX idx_productores_departamento ON productores(departamento);  
CREATE INDEX idx_productores_asociacion   ON productores(asociacion_id);
```

-- TABLA: compradores

```
CREATE TABLE compradores (  
    id          SERIAL      PRIMARY KEY,  
    usuario_id  INTEGER     NOT NULL UNIQUE REFERENCES usuarios(id) ON DELETE  
CASCADE,  
    razon_social VARCHAR(200),  
    nit         VARCHAR(20)  UNIQUE,  
    municipio   VARCHAR(100),  
    departamento VARCHAR(100),  
    tipo_comprador VARCHAR(50),  
    calificacion DECIMAL(3,2) NOT NULL DEFAULT 0.00,  
    created_at  TIMESTAMP    NOT NULL DEFAULT NOW()  
);
```

-- TABLA: productos

```
CREATE TABLE productos (  
    id          SERIAL      PRIMARY KEY,  
    nombre      VARCHAR(150) NOT NULL,  
    categoria   VARCHAR(100),  
    unidad_medida VARCHAR(30) NOT NULL,  
    descripcion TEXT,  
    activo      BOOLEAN     NOT NULL DEFAULT TRUE,  
    created_at  TIMESTAMP    NOT NULL DEFAULT NOW()  
);
```

```
CREATE INDEX idx_productos_categoria ON productos(categoria);
```

-- TABLA: publicaciones

```
CREATE TABLE publicaciones (  
  id SERIAL PRIMARY KEY,  
  productor_id INTEGER NOT NULL REFERENCES productores(id),  
  producto_id INTEGER NOT NULL REFERENCES productos(id),  
  titulo VARCHAR(200) NOT NULL,  
  descripcion TEXT,  
  cantidad_disponible DECIMAL(10,2) NOT NULL,  
  precio_unitario DECIMAL(10,2) NOT NULL,  
  unidad_medida VARCHAR(30) NOT NULL,  
  municipio VARCHAR(100),  
  departamento VARCHAR(100),  
  fecha_cosecha DATE,  
  estado estado_publicacion NOT NULL DEFAULT 'activa',  
  imagen_url VARCHAR(500),  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

```
CREATE INDEX idx_publicaciones_estado ON publicaciones(estado);  
CREATE INDEX idx_publicaciones_departamento ON publicaciones(departamento);  
CREATE INDEX idx_publicaciones_producto ON publicaciones(producto_id);  
CREATE INDEX idx_publicaciones_productor ON publicaciones(productor_id);  
CREATE INDEX idx_publicaciones_precio ON publicaciones(precio_unitario);
```

-- TABLA: negociaciones

```
CREATE TABLE negociaciones (  
  id SERIAL PRIMARY KEY,  
  publicacion_id INTEGER NOT NULL REFERENCES publicaciones(id),  
  comprador_id INTEGER NOT NULL REFERENCES compradores(id),  
  productor_id INTEGER NOT NULL REFERENCES productores(id),  
  cantidad_solicitada DECIMAL(10,2) NOT NULL,  
  precio_acordado DECIMAL(10,2),  
  estado estado_negociacion NOT NULL DEFAULT 'pendiente',  
  condiciones TEXT,  
  fecha_entrega_acordada DATE,  
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMP NOT NULL DEFAULT NOW()  
);
```

```
CREATE INDEX idx_negociaciones_comprador ON negociaciones(comprador_id);  
CREATE INDEX idx_negociaciones_productor ON negociaciones(productor_id);  
CREATE INDEX idx_negociaciones_estado ON negociaciones(estado);  
CREATE INDEX idx_negociaciones_publicacion ON negociaciones(publicacion_id);
```

```
-- TABLA: mensajes
CREATE TABLE mensajes (
  id          SERIAL   PRIMARY KEY,
  negociacion_id INTEGER NOT NULL REFERENCES negociaciones(id) ON DELETE
CASCADE,
  remitente_id INTEGER NOT NULL REFERENCES usuarios(id),
  contenido   TEXT     NOT NULL,
  leido       BOOLEAN  NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX idx_mensajes_negociacion ON mensajes(negociacion_id);
CREATE INDEX idx_mensajes_remitente   ON mensajes(remitente_id);
```

```
-- TABLA: entregas
```

```
CREATE TABLE entregas (
  id          SERIAL   PRIMARY KEY,
  negociacion_id INTEGER NOT NULL UNIQUE REFERENCES negociaciones(id),
  fecha_programada DATE,
  fecha_realizada DATE,
  lugar_entrega VARCHAR(300),
  estado       estado_entrega NOT NULL DEFAULT 'pendiente',
  notas        TEXT,
  created_at   TIMESTAMP     NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMP     NOT NULL DEFAULT NOW()
);
```

```
CREATE INDEX idx_entregas_estado ON entregas(estado);
```

```
-- TABLA: confirmaciones_entrega
```

```
CREATE TABLE confirmaciones_entrega (
  id          SERIAL   PRIMARY KEY,
  entrega_id   INTEGER NOT NULL REFERENCES entregas(id) ON DELETE
CASCADE,
  usuario_id   INTEGER NOT NULL REFERENCES usuarios(id),
  rol_confirmador VARCHAR(20) NOT NULL,
  confirmado   BOOLEAN  NOT NULL DEFAULT FALSE,
  observaciones TEXT,
  fecha_confirmacion TIMESTAMP,
  UNIQUE(entrega_id, usuario_id)
);
```

```
CREATE INDEX idx_confirmaciones_entrega ON confirmaciones_entrega(entrega_id);
```

```
-- TABLA: pagos
```

```
CREATE TABLE pagos (
```

```

id          SERIAL    PRIMARY KEY,
negociacion_id INTEGER NOT NULL REFERENCES negociaciones(id),
monto       DECIMAL(12,2) NOT NULL,
metodo_pago VARCHAR(50) NOT NULL,
referencia  VARCHAR(200),
estado      estado_pago NOT NULL DEFAULT 'pendiente',
fecha_pago  DATE,
registrado_por INTEGER REFERENCES usuarios(id),
notas       TEXT,
created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_pagos_negociacion ON pagos(negociacion_id);
CREATE INDEX idx_pagos_estado     ON pagos(estado);

```

-- TABLA: notificaciones

```

CREATE TABLE notificaciones (
  id          SERIAL    PRIMARY KEY,
  usuario_id  INTEGER NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
  tipo        VARCHAR(50) NOT NULL,
  titulo      VARCHAR(200) NOT NULL,
  mensaje     TEXT      NOT NULL,
  leida       BOOLEAN NOT NULL DEFAULT FALSE,
  referencia_id INTEGER,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

```

```

CREATE INDEX idx_notificaciones_usuario ON notificaciones(usuario_id);
CREATE INDEX idx_notificaciones_leida  ON notificaciones(leida);

```

-- TRIGGER

```

CREATE OR REPLACE FUNCTION fn_update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_usuarios_updated_at
  BEFORE UPDATE ON usuarios
  FOR EACH ROW EXECUTE FUNCTION fn_update_timestamp();

```

```

CREATE TRIGGER trg_publicaciones_updated_at
  BEFORE UPDATE ON publicaciones
  FOR EACH ROW EXECUTE FUNCTION fn_update_timestamp();

```

```

CREATE TRIGGER trg_negociaciones_updated_at
  BEFORE UPDATE ON negociaciones

```

```

FOR EACH ROW EXECUTE FUNCTION fn_update_timestamp();

CREATE TRIGGER trg_entregas_updated_at
BEFORE UPDATE ON entregas
FOR EACH ROW EXECUTE FUNCTION fn_update_timestamp();

CREATE OR REPLACE FUNCTION fn_verificar_doble_confirmacion()
RETURNS TRIGGER AS $$
DECLARE
    v_total_confirmaciones INTEGER;
    v_negociacion_id        INTEGER;
BEGIN
    IF NEW.confirmado = TRUE THEN
        SELECT COUNT(*) INTO v_total_confirmaciones
        FROM confirmaciones_entrega
        WHERE entrega_id = NEW.entrega_id AND confirmado = TRUE;

        IF v_total_confirmaciones >= 2 THEN
            UPDATE entregas
            SET estado = 'entregado', fecha_realizada = NOW()
            WHERE id = NEW.entrega_id;

            SELECT negociacion_id INTO v_negociacion_id
            FROM entregas WHERE id = NEW.entrega_id;

            UPDATE negociaciones
            SET estado = 'completada'
            WHERE id = v_negociacion_id;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_doble_confirmacion
AFTER INSERT OR UPDATE ON confirmaciones_entrega
FOR EACH ROW EXECUTE FUNCTION fn_verificar_doble_confirmacion();

-- VISTAS
-- Vista: publicaciones activas con datos del productor y producto
CREATE OR REPLACE VIEW v_publicaciones_activas AS
SELECT
    p.id,
    p.titulo,
    p.precio_unitario,
    p.cantidad_disponible,
    p.unidad_medida,
    p.departamento,
    p.municipio,

```

```

    p.fecha_cosecha,
    p.imagen_url,
    p.created_at,
    pr.nombre      AS nombre_producto,
    pr.categoria  AS categoria_producto,
    u.nombre      AS nombre_productor,
    prod.calificacion AS calificacion_productor
FROM publicaciones p
JOIN productos pr  ON p.producto_id = pr.id
JOIN productores prod ON p.productor_id = prod.id
JOIN usuarios u    ON prod.usuario_id = u.id
WHERE p.estado = 'activa';

-- Vista: resumen de negociaciones con participantes
CREATE OR REPLACE VIEW v_negociaciones_resumen AS
SELECT
    n.id,
    n.estado,
    n.cantidad_solicitada,
    n.precio_acordado,
    n.fecha_entrega_acordada,
    n.created_at,
    p.titulo      AS titulo_publicacion,
    uc.nombre     AS nombre_comprador,
    up.nombre     AS nombre_productor
FROM negociaciones n
JOIN publicaciones p ON n.publicacion_id = p.id
JOIN compradores c  ON n.comprador_id  = c.id
JOIN usuarios uc    ON c.usuario_id    = uc.id
JOIN productores pr ON n.productor_id  = pr.id
JOIN usuarios up    ON pr.usuario_id   = up.id;
-- FIN DEL SCRIPT

```